

```
In [20]: # Fill in student ID and name
#
student_id = "s225009113"
student_first_last_name = "Devindi"
print(student_id, student_first_last_name)
```

s225009113 Devindi

```
In [22]: import firebase_admin

# Check if default app exists, else initialize
if not firebase_admin._apps:
    cred_obj = firebase_admin.credentials.Certificate(
        "C:/Users/Devindi Amarasinghe/Downloads/devindi-895be-firebase-adminsdk-fbs
    )
    default_app = firebase_admin.initialize_app(cred_obj, {
        "databaseURL": "https://devindi-895be-default-rtdb.firebaseio.com/"
    })
else:
    default_app = firebase_admin.get_app() # reuse existing app

print("Firebase app ready!")
```

Firebase app ready!

```
In [23]: """
        Firebase Realtime database demonstration.

        @Ahsan Habib
        School of IT,
        Deakin University, Australia.
        """

# Install Libraris, if not yet.
! pip install firebase_admin pandas

import firebase_admin

databaseURL = 'https://devindi-895be-default-rtdb.firebaseio.com/'
cred_obj = firebase_admin.credentials.Certificate(
    'devindi-895be-firebase-adminsdk-fbsvc-4b842e80e5.json'
)
default_app = firebase_admin.initialize_app(cred_obj, {
    'databaseURL': databaseURL
})
```

Requirement already satisfied: firebase_admin in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (7.1.0)

Requirement already satisfied: pandas in c:\users\devindi amarasinghe\appdata\roaming\python\python312\site-packages (2.3.1)

Requirement already satisfied: cachecontrol>=0.14.3 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from firebase_admin) (0.14.3)

Requirement already satisfied: google-api-core<3.0.0dev,>=2.25.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (2.25.1)

Requirement already satisfied: google-cloud-firestore>=2.21.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from firebase_admin) (2.21.0)

Requirement already satisfied: google-cloud-storage>=3.1.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from firebase_admin) (3.3.1)

Requirement already satisfied: pyjwt>=2.10.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pyjwt[crypto]>=2.10.1->firebase_admin) (2.10.1)

Requirement already satisfied: httpx==0.28.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx[http2]==0.28.1->firebase_admin) (0.28.1)

Requirement already satisfied: anyio in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (4.2.0)

Requirement already satisfied: certifi in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (2025.1.31)

Requirement already satisfied: httpcore==1.* in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (1.0.2)

Requirement already satisfied: idna in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (3.7)

Requirement already satisfied: h2<5,>=3 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx[http2]==0.28.1->firebase_admin) (4.3.0)

Requirement already satisfied: h11<0.15,>=0.13 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpcore==1.*->httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (0.14.0)

Requirement already satisfied: numpy>=1.26.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas) (1.26.4)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas) (2024.1)

Requirement already satisfied: tzdata>=2022.7 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas) (2023.3)

Requirement already satisfied: requests>=2.16.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from cachecontrol>=0.14.3->firebase_admin) (2.32.3)

Requirement already satisfied: msgpack<2.0.0,>=0.5.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from cachecontrol>=0.14.3->firebase_admin) (1.0.3)

Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.56.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.70.0)

Requirement already satisfied: protobuf!=3.20.0,!3.20.1,!4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<7.0.0,>=3.19.5 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin)

n) (6.32.0)

Requirement already satisfied: proto-plus<2.0.0,>=1.22.3 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.26.1)

Requirement already satisfied: google-auth<3.0.0,>=2.14.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (2.39.0)

Requirement already satisfied: grpcio<2.0.0,>=1.33.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.74.0)

Requirement already satisfied: grpcio-status<2.0.0,>=1.33.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.74.0)

Requirement already satisfied: google-cloud-core<3.0.0,>=1.4.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-cloud-firestore>=2.21.0->firebase_admin) (2.4.3)

Requirement already satisfied: google-resumable-media<3.0.0,>=2.7.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-cloud-storage>=3.1.1->firebase_admin) (2.7.2)

Requirement already satisfied: google-crc32c<2.0.0,>=1.1.3 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-cloud-storage>=3.1.1->firebase_admin) (1.7.1)

Requirement already satisfied: cryptography>=3.4.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pyjwt[crypto]>=2.10.1->firebase_admin) (43.0.0)

Requirement already satisfied: six>=1.5 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)

Requirement already satisfied: cffi>=1.12 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from cryptography>=3.4.0->pyjwt[crypto]>=2.10.1->firebase_admin) (1.17.1)

Requirement already satisfied: cachetools<6.0,>=2.0.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (5.3.3)

Requirement already satisfied: pyasn1-modules>=0.2.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (0.2.8)

Requirement already satisfied: rsa<5,>=3.1.4 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (4.9.1)

Requirement already satisfied: hyperframe<7,>=6.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from h2<5,>=3->httpx[http2]==0.28.1->firebase_admin) (6.1.0)

Requirement already satisfied: hpack<5,>=4.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from h2<5,>=3->httpx[http2]==0.28.1->firebase_admin) (4.1.0)

Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from requests>=2.16.0->cachecontrol>=0.14.3->firebase_admin) (3.3.2)

Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from requests>=2.16.0->cachecontrol>=0.14.3->firebase_admin) (2.2.3)

Requirement already satisfied: sniffio>=1.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from anyio->httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (1.3.0)

Requirement already satisfied: pycparser in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from cffi>=1.12->cryptography>=3.4.0->pyjwt[crypto]>=2.10.1->firebase_admin) (2.21)

Requirement already satisfied: pyasn1<0.5.0,>=0.4.6 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pyasn1-modules>=0.2.1->google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (0.4.8)

ValueError Traceback (most recent call last)

Cell In[23], line 18

```
14 databaseURL = 'https://devindi-895be-default-rtdb.firebaseio.com/'
15 cred_obj = firebase_admin.credentials.Certificate(
16     'devindi-895be-firebase-adminsdk-fbsvc-4b842e80e5.json'
17 )
--> 18 default_app = firebase_admin.initialize_app(cred_obj, {
19     'databaseURL': databaseURL
20 })
```

File ~\1st sem applications\Lib\site-packages\firebase_admin__init__.py:74, in initialize_app(credential, options, name)

```
71     return app
73 if name == _DEFAULT_APP_NAME:
--> 74     raise ValueError((
75         'The default Firebase app already exists. This means you called '
76         'initialize_app() more than once without providing an app name as '
77         'the second argument. In most cases you only need to call '
78         'initialize_app() once. But if you do want to initialize multiple '
79         'apps, pass a second argument to initialize_app() to give each app '
80         'a unique name.'))
82 raise ValueError(
83     f'Firebase app named "{name}" already exists. This means you called '
84     'initialize_app() more than once with the same app name as the '
85     'second argument. Make sure you provide a unique name every time '
86     'you call initialize_app().')
```

ValueError: The default Firebase app already exists. This means you called initialize_app() more than once without providing an app name as the second argument. In most cases you only need to call initialize_app() once. But if you do want to initialize multiple apps, pass a second argument to initialize_app() to give each app a unique name.

In [32]: `from firebase_admin import db`

```
# Reference to the root of the database
ref = db.reference("/")

# JSON format data to set
data = {
    "Book1": {
        "Title": "The Fellowship of the Ring",
        "Author": "J.R.R. Tolkien",
        "Genre": "Epic fantasy",
        "Price": 100
```



```

    },
    "Book2": {
      "Title": "The Two Towers",
      "Author": "J.R.R. Tolkien",
      "Genre": "Epic fantasy",
      "Price": 100
    },
    "Book3": {
      "Title": "The Return of the King",
      "Author": "J.R.R. Tolkien",
      "Genre": "Epic fantasy",
      "Price": 100
    },
    "Book4": {
      "Title": "Brida",
      "Author": "Paulo Coelho",
      "Genre": "Fiction",
      "Price": 100
    }
  }
}

# Set data at the root (overwrites existing data)
ref.set(data)

# Get all data from the root
books = ref.get() # <-- assign the reference to 'books'
print(books)
print(type(books))

# Print each item separately
for key, value in books.items():
    print(f"{key}: {value}")

# Query a specific book (Book1)
book1_ref = db.reference("/Book1")
book1 = book1_ref.get()
print(book1)

```

```

{'Book1': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}, 'Book2': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'}, 'Book3': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'}, 'Book4': {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}}
<class 'dict'>
Book1: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}
Book2: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'}
Book3: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'}
Book4: {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}
{'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}

```

```
In [33]: # Write using push() function
# Note that a set() is called on top of push()
#
ref = db.reference("/")
ref.set({
    "Books":
        {
            "Best_Sellers": -1
        }
})

ref = db.reference("/Books/Best_Sellers")

for key, value in data.items():
    ref.push().set(value)
```

```
In [26]: # Update data
#
# Requirement: The price of the books by
# J. R. R. Tolkien is reduced to 80 units to
# offer a discount.
#
ref = db.reference("/Books/Best_Sellers/")
best_sellers = ref.get()
print(best_sellers)
for key, value in best_sellers.items():
    if(value["Author"] == "J.R.R. Tolkien"):
        value["Price"] = 90
        ref.child(key).update({"Price":80})
```

```
{'-OZ9hTi2jTIti0bga8fT': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}, '-OZ9hTpi1WpxIztILW4n': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'}, '-OZ9hTwuzo0CC6_V3XVC': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'}, '-OZ9hUG-vg32_xrZQw6M': {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}}
```

```
In [34]: # Let's delete all best seller books
# with J.R.R. Tolkien as the author.
#
ref = db.reference("/Books/Best_Sellers")

for key, value in best_sellers.items():
    if(value["Author"] == "J.R.R. Tolkien"):
        ref.child(key).set({})
```

```
In [35]: # Delete all best_seller data.
#
ref = db.reference("/Books/Best_Sellers/")
best_sellers = ref.get()
print(best_sellers)
print(type(best_sellers))
```

```
{'-OZ9j0j8wnQviwCAI21F': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}, '-OZ9j0rjJfQRFvxGPAD3': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'}, '-OZ9j1-CiFZophN-IyHi': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'}, '-OZ9j17YoflmD2rWx0pL': {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}}
<class 'dict'>
```

```
In [36]: ref = db.reference("/Books/Best_Sellers")
ref.set({})
```

```
In [37]: import os

folder_path = r"D:\Deakin 2nd Sem\sit 225 Data Capture Technologies"
os.makedirs(folder_path, exist_ok=True) # exist_ok=True prevents error if folder exists

print(f"Folder created at: {folder_path}")
```

Folder created at: D:\Deakin 2nd Sem\sit 225 Data Capture Technologies

```
In [39]: import os
import subprocess

# Path to your local repository
repo_path = r"D:\Deakin 2nd Sem\sit 225 Data Capture Technologies"

# Make sure the folder exists
os.makedirs(repo_path, exist_ok=True)

# Construct the MiKTeX command
cmd = [
    "miktexsetup",
    "--verbose",
    f"--local-package-repository={repo_path}",
    "--package-set=complete",
    "download"
]

# Option 1: Using subprocess (recommended)
subprocess.run(cmd, check=True)

# Option 2: Using os.system (simpler)
# os.system(f'miktexsetup --verbose --local-package-repository="{repo_path}" --package-set=complete')

print("MiKTeX packages download command executed!")
```

```

-----
CalledProcessError                                Traceback (most recent call last)
Cell In[39], line 20
    11 cmd = [
    12     "miktexsetup",
    13     "--verbose",
    (...)
    16     "download"
    17 ]
    19 # Option 1: Using subprocess (recommended)
--> 20 subprocess.run(cmd, check=True)
    22 # Option 2: Using os.system (simpler)
    23 # os.system(f'miktexsetup --verbose --local-package-repository="{repo_path}"
--package-set=complete download')
    25 print("MiKTeX packages download command executed!")

File ~\1st sem applications\Lib\subprocess.py:571, in run(input, capture_output, tim
eout, check, *popenargs, **kwargs)
    569     retcode = process.poll()
    570     if check and retcode:
--> 571         raise CalledProcessError(retcode, process.args,
    572                                     output=stdout, stderr=stderr)
    573     return CompletedProcess(process.args, retcode, stdout, stderr)

CalledProcessError: Command '['miktexsetup', '--verbose', '--local-package-repositor
y=D:\\Deakin 2nd Sem\\sit 225 Data Capture Technologies', '--package-set=complete',
'download']' returned non-zero exit status 1.

```

```

In [41]: sensorData={
    "sensors": [
        {
            "sensor_name": "DHT22",
            "readings": [
                {
                    "timestamp": "2025-09-02T10:00:00",
                    "temperature": 25.3,
                    "humidity": 60
                },
                {
                    "timestamp": "2025-09-02T10:05:00",
                    "temperature": 25.5,
                    "humidity": 61
                }
            ]
        },
        {
            "sensor_name": "Accelerometer",
            "readings": [
                {
                    "timestamp": "2025-09-02T10:00:00",
                    "x": 0.12,
                    "y": 0.01,
                    "z": 0.98
                },
                {
                    "timestamp": "2025-09-02T10:05:00",

```

```

        "x": 0.15,
        "y": 0.03,
        "z": 0.97
    }
]
}
]
}

```

```

In [43]: import random
from datetime import datetime
from firebase_admin import db

# Reference to the DHT22 node in your Firebase database
ref = db.reference("/DHT22")

# Function to generate random DHT22 data
def generate_dht22_data():
    return {
        "timestamp": datetime.now().isoformat(),
        "temperature": round(random.uniform(20.0, 30.0), 2), # Random temp 20-30°C
        "humidity": round(random.uniform(40.0, 70.0), 2)      # Random humidity 40-
    }

# Insert 5 random readings into the database
for _ in range(5):
    reading = generate_dht22_data()
    ref.push(reading) # Push creates a unique key for each reading

# Query all data from the DHT22 node
all_data = ref.get()

# Print all data
print("All DHT22 Sensor Data:")
for key, value in all_data.items():
    print(f"{key}: {value}")

```

```

All DHT22 Sensor Data:
-OZA-1LMQUiQx-AG11KL: {'humidity': 42.63, 'temperature': 27.71, 'timestamp': '2025-09-03T01:19:34.612307'}
-OZA-1P0S_F-A5xMmEAz: {'humidity': 45.05, 'temperature': 25.4, 'timestamp': '2025-09-03T01:19:35.517415'}
-OZA-1Sl8AmS9XRcwoN4: {'humidity': 47.5, 'temperature': 25.48, 'timestamp': '2025-09-03T01:19:35.760350'}
-OZA-1WZD_ezCHyY-Pno: {'humidity': 69.09, 'temperature': 22.03, 'timestamp': '2025-09-03T01:19:36.000975'}
-OZA-1_OZpGpGtPoc3il: {'humidity': 67.05, 'temperature': 28.8, 'timestamp': '2025-09-03T01:19:36.247323'}

```

```

In [45]: import random
from datetime import datetime
from firebase_admin import db

# Reference to the SR04 node
ref = db.reference("/SR04")

```

```

# Function to generate random SR04 data (distance in cm)
def generate_sr04_data():
    return {
        "timestamp": datetime.now().isoformat(),
        "distance_cm": round(random.uniform(2.0, 400.0), 2) # SR04 range: 2cm - 400cm
    }

# Insert 5 random readings into the database
for _ in range(5):
    reading = generate_sr04_data()
    ref.push(reading) # push creates a unique key for each reading

# Query all data from the SR04 node
all_data = ref.get()

# Print all data
print("All SR04 Sensor Data:")
for key, value in all_data.items():
    print(f"{key}: {value}")

```

All SR04 Sensor Data:

```

-OZA0SbMr59Qo0Bgs9gy: {'distance_cm': 303.95, 'timestamp': '2025-09-03T01:22:36.246291'}
-OZA0V4eFJtrahKrRLS_: {'distance_cm': 292.4, 'timestamp': '2025-09-03T01:22:36.858421'}
-OZA0V9Swz3T6IATCK60: {'distance_cm': 108.88, 'timestamp': '2025-09-03T01:22:47.002349'}
-OZA0VEp4M4vJG6g1KTX: {'distance_cm': 260.41, 'timestamp': '2025-09-03T01:22:47.307943'}
-OZA0VJkZcykOSIq6BvA: {'distance_cm': 174.96, 'timestamp': '2025-09-03T01:22:47.649511'}
-OZA0XRwp3NzQVJyt_rn: {'distance_cm': 216.2, 'timestamp': '2025-09-03T01:22:56.375033'}
-OZA0XWcrnZBvvivjkhy: {'distance_cm': 254.6, 'timestamp': '2025-09-03T01:22:56.679739'}
-OZA0XaLckpJiYoD0q02: {'distance_cm': 71.62, 'timestamp': '2025-09-03T01:22:56.983011'}
-OZA0Xf6Grgmcu1w98QJ: {'distance_cm': 214.37, 'timestamp': '2025-09-03T01:22:57.282450'}
-OZA0Xjn-xIoJzUtcL03: {'distance_cm': 119.03, 'timestamp': '2025-09-03T01:22:57.589014'}

```

In []:

```
In [20]: # Fill in student ID and name
#
student_id = "s225009113"
student_first_last_name = "Devindi"
print(student_id, student_first_last_name)
```

s225009113 Devindi

```
In [22]: import firebase_admin

# Check if default app exists, else initialize
if not firebase_admin._apps:
    cred_obj = firebase_admin.credentials.Certificate(
        "C:/Users/Devindi Amarasinghe/Downloads/devindi-895be-firebase-adminsdk-fbs
    )
    default_app = firebase_admin.initialize_app(cred_obj, {
        "databaseURL": "https://devindi-895be-default-rtdb.firebaseio.com/"
    })
else:
    default_app = firebase_admin.get_app() # reuse existing app

print("Firebase app ready!")
```

Firebase app ready!

```
In [23]: """
        Firebase Realtime database demonstration.

        @Ahsan Habib
        School of IT,
        Deakin University, Australia.
        """

# Install Libraris, if not yet.
! pip install firebase_admin pandas

import firebase_admin

databaseURL = 'https://devindi-895be-default-rtdb.firebaseio.com/'
cred_obj = firebase_admin.credentials.Certificate(
    'devindi-895be-firebase-adminsdk-fbsvc-4b842e80e5.json'
)
default_app = firebase_admin.initialize_app(cred_obj, {
    'databaseURL': databaseURL
})
```


Requirement already satisfied: firebase_admin in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (7.1.0)

Requirement already satisfied: pandas in c:\users\devindi amarasinghe\appdata\roaming\python\python312\site-packages (2.3.1)

Requirement already satisfied: cachecontrol>=0.14.3 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from firebase_admin) (0.14.3)

Requirement already satisfied: google-api-core<3.0.0dev,>=2.25.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (2.25.1)

Requirement already satisfied: google-cloud-firestore>=2.21.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from firebase_admin) (2.21.0)

Requirement already satisfied: google-cloud-storage>=3.1.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from firebase_admin) (3.3.1)

Requirement already satisfied: pyjwt>=2.10.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pyjwt[crypto]>=2.10.1->firebase_admin) (2.10.1)

Requirement already satisfied: httpx==0.28.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx[http2]==0.28.1->firebase_admin) (0.28.1)

Requirement already satisfied: anyio in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (4.2.0)

Requirement already satisfied: certifi in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (2025.1.31)

Requirement already satisfied: httpcore==1.* in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (1.0.2)

Requirement already satisfied: idna in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (3.7)

Requirement already satisfied: h2<5,>=3 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpx[http2]==0.28.1->firebase_admin) (4.3.0)

Requirement already satisfied: h11<0.15,>=0.13 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from httpcore==1.*->httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (0.14.0)

Requirement already satisfied: numpy>=1.26.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas) (1.26.4)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas) (2024.1)

Requirement already satisfied: tzdata>=2022.7 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas) (2023.3)

Requirement already satisfied: requests>=2.16.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from cachecontrol>=0.14.3->firebase_admin) (2.32.3)

Requirement already satisfied: msgpack<2.0.0,>=0.5.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from cachecontrol>=0.14.3->firebase_admin) (1.0.3)

Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.56.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.70.0)

Requirement already satisfied: protobuf!=3.20.0,!3.20.1,!4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<7.0.0,>=3.19.5 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (3.20.3)

n) (6.32.0)

Requirement already satisfied: proto-plus<2.0.0,>=1.22.3 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.26.1)

Requirement already satisfied: google-auth<3.0.0,>=2.14.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (2.39.0)

Requirement already satisfied: grpcio<2.0.0,>=1.33.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.74.0)

Requirement already satisfied: grpcio-status<2.0.0,>=1.33.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.74.0)

Requirement already satisfied: google-cloud-core<3.0.0,>=1.4.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-cloud-firestore>=2.21.0->firebase_admin) (2.4.3)

Requirement already satisfied: google-resumable-media<3.0.0,>=2.7.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-cloud-storage>=3.1.1->firebase_admin) (2.7.2)

Requirement already satisfied: google-crc32c<2.0.0,>=1.1.3 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-cloud-storage>=3.1.1->firebase_admin) (1.7.1)

Requirement already satisfied: cryptography>=3.4.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pyjwt[crypto]>=2.10.1->firebase_admin) (43.0.0)

Requirement already satisfied: six>=1.5 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)

Requirement already satisfied: cffi>=1.12 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from cryptography>=3.4.0->pyjwt[crypto]>=2.10.1->firebase_admin) (1.17.1)

Requirement already satisfied: cachetools<6.0,>=2.0.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (5.3.3)

Requirement already satisfied: pyasn1-modules>=0.2.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (0.2.8)

Requirement already satisfied: rsa<5,>=3.1.4 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (4.9.1)

Requirement already satisfied: hyperframe<7,>=6.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from h2<5,>=3->httpx[http2]==0.28.1->firebase_admin) (6.1.0)

Requirement already satisfied: hpack<5,>=4.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from h2<5,>=3->httpx[http2]==0.28.1->firebase_admin) (4.1.0)

Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from requests>=2.16.0->cachecontrol>=0.14.3->firebase_admin) (3.3.2)

Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from requests>=2.16.0->cachecontrol>=0.14.3->firebase_admin) (2.2.3)

Requirement already satisfied: sniffio>=1.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from anyio->httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (1.3.0)

Requirement already satisfied: pycparser in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from cffi>=1.12->cryptography>=3.4.0->pyjwt[crypto]>=2.10.1->firebase_admin) (2.21)

Requirement already satisfied: pyasn1<0.5.0,>=0.4.6 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pyasn1-modules>=0.2.1->google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (0.4.8)

ValueError Traceback (most recent call last)

Cell In[23], line 18

```
14 databaseURL = 'https://devindi-895be-default-rtdb.firebaseio.com/'
15 cred_obj = firebase_admin.credentials.Certificate(
16     'devindi-895be-firebase-adminsdk-fbsvc-4b842e80e5.json'
17 )
--> 18 default_app = firebase_admin.initialize_app(cred_obj, {
19     'databaseURL': databaseURL
20 })
```

File ~\1st sem applications\Lib\site-packages\firebase_admin__init__.py:74, in initialize_app(credential, options, name)

```
71     return app
73 if name == _DEFAULT_APP_NAME:
--> 74     raise ValueError((
75         'The default Firebase app already exists. This means you called '
76         'initialize_app() more than once without providing an app name as '
77         'the second argument. In most cases you only need to call '
78         'initialize_app() once. But if you do want to initialize multiple '
79         'apps, pass a second argument to initialize_app() to give each app '
80         'a unique name.'))
82 raise ValueError(
83     f'Firebase app named "{name}" already exists. This means you called '
84     'initialize_app() more than once with the same app name as the '
85     'second argument. Make sure you provide a unique name every time '
86     'you call initialize_app().')
```

ValueError: The default Firebase app already exists. This means you called initialize_app() more than once without providing an app name as the second argument. In most cases you only need to call initialize_app() once. But if you do want to initialize multiple apps, pass a second argument to initialize_app() to give each app a unique name.

In [32]: **from** firebase_admin **import** db

```
# Reference to the root of the database
ref = db.reference("/")

# JSON format data to set
data = {
    "Book1": {
        "Title": "The Fellowship of the Ring",
        "Author": "J.R.R. Tolkien",
        "Genre": "Epic fantasy",
        "Price": 100
```

```

    },
    "Book2": {
      "Title": "The Two Towers",
      "Author": "J.R.R. Tolkien",
      "Genre": "Epic fantasy",
      "Price": 100
    },
    "Book3": {
      "Title": "The Return of the King",
      "Author": "J.R.R. Tolkien",
      "Genre": "Epic fantasy",
      "Price": 100
    },
    "Book4": {
      "Title": "Brida",
      "Author": "Paulo Coelho",
      "Genre": "Fiction",
      "Price": 100
    }
  }
}

# Set data at the root (overwrites existing data)
ref.set(data)

# Get all data from the root
books = ref.get() # <-- assign the reference to 'books'
print(books)
print(type(books))

# Print each item separately
for key, value in books.items():
    print(f"{key}: {value}")

# Query a specific book (Book1)
book1_ref = db.reference("/Book1")
book1 = book1_ref.get()
print(book1)

```

```

{'Book1': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}, 'Book2': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'}, 'Book3': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'}, 'Book4': {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}}
<class 'dict'>
Book1: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}
Book2: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'}
Book3: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'}
Book4: {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}
{'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}

```

```
In [33]: # Write using push() function
# Note that a set() is called on top of push()
#
ref = db.reference("/")
ref.set({
    "Books":
        {
            "Best_Sellers": -1
        }
})

ref = db.reference("/Books/Best_Sellers")

for key, value in data.items():
    ref.push().set(value)
```

```
In [26]: # Update data
#
# Requirement: The price of the books by
# J. R. R. Tolkien is reduced to 80 units to
# offer a discount.
#
ref = db.reference("/Books/Best_Sellers/")
best_sellers = ref.get()
print(best_sellers)
for key, value in best_sellers.items():
    if(value["Author"] == "J.R.R. Tolkien"):
        value["Price"] = 90
        ref.child(key).update({"Price":80})
```

```
{'-OZ9hTi2jTIti0bga8fT': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}, '-OZ9hTpi1WpxIztILW4n': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'}, '-OZ9hTwuzo0CC6_V3XVC': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'}, '-OZ9hUG-vg32_xrZQw6M': {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}}
```

```
In [34]: # Let's delete all best seller books
# with J.R.R. Tolkien as the author.
#
ref = db.reference("/Books/Best_Sellers")

for key, value in best_sellers.items():
    if(value["Author"] == "J.R.R. Tolkien"):
        ref.child(key).set({})
```

```
In [35]: # Delete all best_seller data.
#
ref = db.reference("/Books/Best_Sellers/")
best_sellers = ref.get()
print(best_sellers)
print(type(best_sellers))
```

```
{'-OZ9j0j8wnQviwCAI21F': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}, '-OZ9j0rjJfQRFvxGPAD3': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'}, '-OZ9j1-CiFZophN-IyHi': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'}, '-OZ9j17YoflmD2rWx0pL': {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}}
<class 'dict'>
```

```
In [36]: ref = db.reference("/Books/Best_Sellers")
ref.set({})
```

```
In [37]: import os

folder_path = r"D:\Deakin 2nd Sem\sit 225 Data Capture Technologies"
os.makedirs(folder_path, exist_ok=True) # exist_ok=True prevents error if folder exists

print(f"Folder created at: {folder_path}")
```

Folder created at: D:\Deakin 2nd Sem\sit 225 Data Capture Technologies

```
In [39]: import os
import subprocess

# Path to your local repository
repo_path = r"D:\Deakin 2nd Sem\sit 225 Data Capture Technologies"

# Make sure the folder exists
os.makedirs(repo_path, exist_ok=True)

# Construct the MiKTeX command
cmd = [
    "miktexsetup",
    "--verbose",
    f"--local-package-repository={repo_path}",
    "--package-set=complete",
    "download"
]

# Option 1: Using subprocess (recommended)
subprocess.run(cmd, check=True)

# Option 2: Using os.system (simpler)
# os.system(f'miktexsetup --verbose --local-package-repository="{repo_path}" --package-set=complete')

print("MiKTeX packages download command executed!")
```

```

-----
CalledProcessError                                Traceback (most recent call last)
Cell In[39], line 20
     11 cmd = [
     12     "miktexsetup",
     13     "--verbose",
    (...)
     16     "download"
     17 ]
     19 # Option 1: Using subprocess (recommended)
--> 20 subprocess.run(cmd, check=True)
     22 # Option 2: Using os.system (simpler)
     23 # os.system(f'miktexsetup --verbose --local-package-repository="{repo_path}"
--package-set=complete download')
     25 print("MiKTeX packages download command executed!")

File ~\1st sem applications\Lib\subprocess.py:571, in run(input, capture_output, tim
eout, check, *popenargs, **kwargs)
     569     retcode = process.poll()
     570     if check and retcode:
--> 571         raise CalledProcessError(retcode, process.args,
     572                                     output=stdout, stderr=stderr)
     573     return CompletedProcess(process.args, retcode, stdout, stderr)

CalledProcessError: Command '['miktexsetup', '--verbose', '--local-package-repositor
y=D:\\Deakin 2nd Sem\\sit 225 Data Capture Technologies', '--package-set=complete',
'download']' returned non-zero exit status 1.

```

```

In [41]: sensorData={
    "sensors": [
        {
            "sensor_name": "DHT22",
            "readings": [
                {
                    "timestamp": "2025-09-02T10:00:00",
                    "temperature": 25.3,
                    "humidity": 60
                },
                {
                    "timestamp": "2025-09-02T10:05:00",
                    "temperature": 25.5,
                    "humidity": 61
                }
            ]
        },
        {
            "sensor_name": "Accelerometer",
            "readings": [
                {
                    "timestamp": "2025-09-02T10:00:00",
                    "x": 0.12,
                    "y": 0.01,
                    "z": 0.98
                },
                {
                    "timestamp": "2025-09-02T10:05:00",

```



```

        "x": 0.15,
        "y": 0.03,
        "z": 0.97
    }
]
}
]
}

```

```

In [43]: import random
from datetime import datetime
from firebase_admin import db

# Reference to the DHT22 node in your Firebase database
ref = db.reference("/DHT22")

# Function to generate random DHT22 data
def generate_dht22_data():
    return {
        "timestamp": datetime.now().isoformat(),
        "temperature": round(random.uniform(20.0, 30.0), 2), # Random temp 20-30°C
        "humidity": round(random.uniform(40.0, 70.0), 2)      # Random humidity 40-
    }

# Insert 5 random readings into the database
for _ in range(5):
    reading = generate_dht22_data()
    ref.push(reading) # Push creates a unique key for each reading

# Query all data from the DHT22 node
all_data = ref.get()

# Print all data
print("All DHT22 Sensor Data:")
for key, value in all_data.items():
    print(f"{key}: {value}")

```

All DHT22 Sensor Data:

```

-OZA-1LMQUiQx-AG11KL: {'humidity': 42.63, 'temperature': 27.71, 'timestamp': '2025-09-03T01:19:34.612307'}
-OZA-1P0S_F-A5xMmEAz: {'humidity': 45.05, 'temperature': 25.4, 'timestamp': '2025-09-03T01:19:35.517415'}
-OZA-1Sl8AmS9XRcwoN4: {'humidity': 47.5, 'temperature': 25.48, 'timestamp': '2025-09-03T01:19:35.760350'}
-OZA-1WZD_ezCHyY-Pno: {'humidity': 69.09, 'temperature': 22.03, 'timestamp': '2025-09-03T01:19:36.000975'}
-OZA-1_OZpGpGtPoc3il: {'humidity': 67.05, 'temperature': 28.8, 'timestamp': '2025-09-03T01:19:36.247323'}

```

```

In [45]: import random
from datetime import datetime
from firebase_admin import db

# Reference to the SR04 node
ref = db.reference("/SR04")

```



```

# Function to generate random SR04 data (distance in cm)
def generate_sr04_data():
    return {
        "timestamp": datetime.now().isoformat(),
        "distance_cm": round(random.uniform(2.0, 400.0), 2) # SR04 range: 2cm - 400cm
    }

# Insert 5 random readings into the database
for _ in range(5):
    reading = generate_sr04_data()
    ref.push(reading) # push creates a unique key for each reading

# Query all data from the SR04 node
all_data = ref.get()

# Print all data
print("All SR04 Sensor Data:")
for key, value in all_data.items():
    print(f"{key}: {value}")

```

All SR04 Sensor Data:

```

-OZA0SbMr59Qo0Bgs9gy: {'distance_cm': 303.95, 'timestamp': '2025-09-03T01:22:36.246291'}
-OZA0V4eFJtrahKrRLS_: {'distance_cm': 292.4, 'timestamp': '2025-09-03T01:22:36.858421'}
-OZA0V9Swz3T6IATCK60: {'distance_cm': 108.88, 'timestamp': '2025-09-03T01:22:47.002349'}
-OZA0VEp4M4vJG6g1KTX: {'distance_cm': 260.41, 'timestamp': '2025-09-03T01:22:47.307943'}
-OZA0VJkZcykOSIq6BvA: {'distance_cm': 174.96, 'timestamp': '2025-09-03T01:22:47.649511'}
-OZA0XRwp3NzQVJyt_rn: {'distance_cm': 216.2, 'timestamp': '2025-09-03T01:22:56.375033'}
-OZA0XWcrnZBvvivjkhy: {'distance_cm': 254.6, 'timestamp': '2025-09-03T01:22:56.679739'}
-OZA0XaLckpJiYoD0q02: {'distance_cm': 71.62, 'timestamp': '2025-09-03T01:22:56.983011'}
-OZA0Xf6Grgmcu1w98QJ: {'distance_cm': 214.37, 'timestamp': '2025-09-03T01:22:57.282450'}
-OZA0Xjn-xIoJzUtcL03: {'distance_cm': 119.03, 'timestamp': '2025-09-03T01:22:57.589014'}

```

In []:

SIT225 Data Capture Technologies

Pass Task: Store data to cloud

Overview

The Firebase Realtime Database is a cloud-hosted NoSQL database that lets you store and sync data between your users in real-time. Data is stored as JSON and synchronized in real-time to every connected client.

Hardware Required

i. Arduino Nano 33 IoT device, ii. USB cable, iii. LSM6DS3 module on the Arduino Nano 33 IoT for Gyroscope data.

Software Required

Python 3.

Pre-requisites: You must do the following before this task

Week 5 activities in the unit site.

Task Objective

In this week, you have learned how to use Firebase real-time database and perform operations such as add/update/query data and listen to live changes to your data. In this task, you will need to record Gyroscope readings from the built-in module of Arduino Nano board and pass data to a Python script through Serial communication (or through Arduino IoT Cloud variable synchronisation with Python). The Python script upload data to Firebase real-time database as soon as it receives any. After enough data is stored to capture interesting patterns, for no less than 10 minutes, query the database for data, format it to save in a CSV file and analyse the data. *You may reuse this data next week, so keep your CSV file handy.*

Steps:

1. Connect the sensor to collect data samples for no less than 30 minutes. Justify your data collection decision in terms of what, when and how long to capture with respect to the phenomena you want to capture with Gyroscope sensor.

To capture meaningful gyroscope data, I connected the LSM6DS3 sensor on the Arduino Nano 33 IoT and collected data continuously for **30 minutes**. The purpose of collecting data for this duration is to capture enough motion patterns, including rotations and orientation changes, to observe trends, periodic motions, and relative changes in the x, y, and z axes. A shorter collection period may miss certain repeating patterns or irregular movements, while 30 minutes provides sufficient time to record a variety of motions that reflect realistic usage scenarios. Data was captured **continuously in real time** at a sampling frequency of **10 Hz (one sample every 100 ms)**, which balances capturing detailed motion data without overloading storage or transmission. This duration and sampling frequency ensure the dataset is large enough to analyze trends, detect peaks and oscillations, and support future processing such as plotting and pattern recognition.

2. Find and report on the data rate of the Gyroscope module and decide how frequently you want to sample data from it. Justify your answer.

Observed Sampling Rate

Both your Python script and Arduino sketch use:

```
time.sleep(0.1) # Python
```

```
delay(100); // Arduino
```

This corresponds to:

- **Sampling interval:** 100 milliseconds
- **Sampling frequency**

So you're currently sampling at **10 Hz**, meaning **10 data points per second**.

Recommended Sampling Frequency

Suggested Rate: **50 Hz**

Justification

Factor	Explanation
Human motion tracking	Most human movements (e.g., walking, gestures) occur below 20 Hz. Sampling at 50 Hz captures these with high fidelity while avoiding aliasing.
Nyquist Criterion	To accurately capture signals up to 25 Hz, you need at least 50 Hz sampling. This

	ensures clean reconstruction of motion signals.
Data volume vs. responsiveness	50 Hz offers a good balance between responsiveness and manageable data size for real-time Firebase uploads.
Sensor capability	Most gyroscope modules like MPU6050 support up to 1000 Hz, so 50 Hz is well within safe operating limits.
Latency-sensitive applications	If you're building gesture recognition, fall detection, or robotics control, 10 Hz may feel sluggish. 50 Hz improves responsiveness.

Final Recommendation

Update delay to 20 ms:

time.sleep(0.02) # Python

delay(20); // Arduino

This sets sampling rate to **50 Hz**, which is ideal for most motion-based applications without overwhelming your database or processing pipeline.

3. Consider the number of data variables per sensor, since you will need to transmit them through the serial communication port (or alternatively, through Arduino IoT Cloud). The number of variables for LSM6DS3 Gyroscope is 3 (x, y, z).

The LSM6DS3 gyroscope returns **3 variables per sample**: angular rate on **X, Y, Z** axes (commonly called gx, gy, gz). Below I show how that affects the serial transmission, give numeric byte-by-byte calculations, and finish with a clear sampling recommendation + example code for sending all three variables with a timestamp.

1) What we must transmit per sample

Per sample we typically want to transmit:

- gx (raw 16-bit or scaled value)
- gy

- gz
- timestamp (Unix seconds or ISO datetime) — optional but recommended

That is **4 variables** if you include a timestamp, otherwise **3 variables**.

2) Raw binary vs ASCII (human readable) — difference matters

- **Raw binary (compact):** each axis often fits in **16 bits (2 bytes)** → 3 axes = **6 bytes**. Add a 4-byte float timestamp → **10 bytes/sample** (very efficient). Requires matching binary parsing on the receiver.
- **ASCII text (easy/debug):** numbers printed as decimal text (e.g. -32768,123,0,2025-09-03T22:45:56\n). ASCII is much longer (human-readable but uses more bandwidth).

I will base calculations on the **ASCII** format because your current code uses CSV Serial.print() which is ASCII.

3) Estimate bytes per ASCII sample (digit-by-digit)

Take a conservative worst-case ASCII line (no spaces), example:

-32768,-32768,-32768,2025-09-03 22:45:56\n

Count characters:

- each axis -32768 → **6 chars** × 3 = **18**
- two commas between axes → **2**
- comma before datetime → **1**
- datetime "2025-09-03 22:45:56" → **19**
- newline \n → **1**

Total ≈ **18 + 2 + 1 + 19 + 1 = 41 bytes** per line.

If you use Unix timestamp (e.g. 1693743956.123) that's ~14 chars instead of 19, reducing total to ~36 bytes. If axis values are smaller (e.g. 1234), bytes drop accordingly.

Use these numbers for capacity checks:

- Conservative ASCII sample ≈ **40 bytes/sample**
 - Smaller ASCII sample ≈ **25–30 bytes/sample**
-

4) Serial capacity (bit/byte math)

Serial link properties:

- Baud = bits per second (bps). Example common values: **9600** or **115200**.
- UART framing typically 1 start + 8 data + 1 stop = **10 bits per byte** → effective bytes/s = baud / 10.

Compute max samples/sec = (baud / 10) / (bytes_per_sample)

Examples (use the conservative 40 bytes/sample):

- **9600 baud:** bytes/s = 9600 / 10 = **960 bytes/s**
samples/s $\approx 960 / 40 = 24 \rightarrow \sim \mathbf{24 \text{ samples/sec}}$ maximum
- **115200 baud:** bytes/s = 115200 / 10 = **11520 bytes/s**
samples/s $\approx 11520 / 40 = 288 \rightarrow \sim \mathbf{288 \text{ samples/sec}}$ maximum

If you use the smaller estimate 25 bytes/sample:

- 9600 $\rightarrow 960 / 25 = 38.4 \rightarrow \sim \mathbf{38 \text{ samples/sec}}$
- 115200 $\rightarrow 11520 / 25 = 460.8 \rightarrow \sim \mathbf{460 \text{ samples/sec}}$

5) Network / Firebase considerations

- Each sample in your Python uploader currently becomes **one Firebase write**.
- Example write counts:
 - **10 Hz** for 30 min $\rightarrow 10 * 60 * 30 = 18,000$ writes
 - **100 Hz** for 30 min $\rightarrow 100 * 60 * 30 = 180,000$ writes
- Frequent individual writes increase latency, possible throttling, and cost. Prefer **batching/aggregation** for cloud upload.

6) Sampling decision (final recommendation + justification)

Recommendation:

- **Sample locally at 100 Hz** (to capture motion up to ~ 50 Hz and give good fidelity for human/device motion) **but do not upload every single sample**. Instead **buffer** and **upload averaged/aggregated data at 10 Hz** (or push batches every 1 second).
- If you cannot buffer locally, choose **10–20 Hz** sampling and upload every sample to Firebase.

Why:

- 3-axis gyro = only three variables, but ASCII transmission and timestamp inflate bytes/sample. At **9600 baud**, streaming 100 Hz ASCII lines is likely to exceed serial capacity.
- Local 100 Hz sampling preserves fidelity (Nyquist for motions up to ~ 50 Hz). Buffering + batch upload avoids flooding Firebase and keeps network usage manageable.
- If you insist on streaming every sample to Firebase at 100 Hz, you must: (a) raise serial to **115200** or higher and (b) ensure your network and Firebase rules/costs allow that write rate.

7) Code examples

Arduino — print gx,gy,gz,unix_timestamp (ASCII CSV)

```
#include <Wire.h>
```

```
#include <LSM6DS3.h> // or appropriate library for LSM6DS3
```

```
LSM6DS3 imu(Wire);
```

```
void setup() {
  Serial.begin(115200); // use 115200 if you plan > ~30 Hz with long lines
  Wire.begin();
  // imu initialization...
}
```

```
void loop() {
  int16_t gx = imu.readGyroX(); // replace with your lib call
  int16_t gy = imu.readGyroY();
  int16_t gz = imu.readGyroZ();

  unsigned long t = millis(); // ms since start
  // send as CSV (gx,gy,gz,timestamp_seconds)
  Serial.print(gx); Serial.print(",");
  Serial.print(gy); Serial.print(",");
  Serial.print(gz); Serial.print(",");
  Serial.println(millis() / 1000.0, 3); // seconds with ms precision

  delay(10); // 10 ms -> ~100 Hz sampling
}
```

Python — buffer 100 Hz samples and upload averaged sample to Firebase every 1 second

```
import serial, time, firebase_admin
from firebase_admin import credentials, db
from statistics import mean

cred = credentials.Certificate("path/to/your-key.json")
firebase_admin.initialize_app(cred, {'databaseURL': 'https://your-db.firebaseio.com'})

ser = serial.Serial("COM3", 115200, timeout=1) # match Arduino baud
buffer = []
start = time.time()

while True:
  line = ser.readline().decode(errors='ignore').strip()
  if not line:
    continue
```

```

try:
    gx, gy, gz, ts = line.split(",")
    buffer.append((int(gx), int(gy), int(gz)))
except:
    continue

# every 1 second push aggregated sample
if time.time() - start >= 1.0:
    if buffer:
        avg_gx = int(mean([b[0] for b in buffer]))
        avg_gy = int(mean([b[1] for b in buffer]))
        avg_gz = int(mean([b[2] for b in buffer]))
        data = {
            'gx': avg_gx,
            'gy': avg_gy,
            'gz': avg_gz,
            'datetime': time.strftime("%Y-%m-%d %H:%M:%S")
        }
        db.reference('gyroscope_data').push(data)
        buffer = []
    start = time.time()

```

- The LSM6DS3 gyroscope provides **3 variables** (gx, gy, gz). When including a timestamp this becomes **4 variables per sample**.
- If you transmit ASCII CSV, expect **~25–40 bytes per sample** (digit-by-digit calculation shown above).
- At **9600 baud**, this supports roughly **~24–38 samples/sec** depending on line length; at **115200 baud**, **~288–460 samples/sec**.
- **Final plan:** sample **locally at 100 Hz** for fidelity, **buffer** samples and **upload aggregated results at 10 Hz (or 1 Hz batches)** to Firebase — this balances signal quality, serial capacity, and network/database load.

4. Write a Python script to continuously run and perform the following actions:
 - a. Receive data following the protocol how data is sent from the Arduino program,

- b. Record your computer's current timestamp and prepare a JSON message with timestamp and x, y, z and add this data sample to Firebase.
- c. Repeat step 4 as long as Arduino is sending data, stop both processes once data collection is done.

```

import serial
import time
from datetime import datetime
import firebase_admin
from firebase_admin import credentials, db

# -----
# 1. Firebase Setup
# -----
# Path to your Firebase service account JSON
cred = credentials.Certificate("C:/Users/Devindi Amarasinghe/Downloads/devindi-895be-
firebase-adminsdk-fbsvc-4b842e80e5.json")
firebase_admin.initialize_app(cred, {
    "databaseURL": "https://devindi-895be-default-rtdb.firebaseio.com/"
})

# Reference to the database node where data will be stored
ref = db.reference('gyroscope_data')

# -----
# 2. Serial Setup
# -----
# Connect to Arduino Serial port (replace 'COM3' with your port)
ser = serial.Serial('COM3', 9600)
time.sleep(2) # Give Serial port time to initialize

# -----
# 3. Receive and Upload Data
# -----
try:
    while True:
        # Read a line from Serial and decode to string
        line = ser.readline().decode('utf-8').strip()

        # Skip CSV header line
        if line.startswith("timestamp_ms"):

```

```

        continue

    # Split the CSV line into components
    parts = line.split(',')
    if len(parts) != 4:
        continue # Skip invalid lines

    timestamp_ms, x, y, z = parts

    try:
        # Prepare JSON object with timestamp and gyroscope values
        data = {
            'timestamp': int(timestamp_ms), # Arduino timestamp in ms
            'x': float(x),
            'y': float(y),
            'z': float(z)
        }

        # Push data to Firebase
        ref.push(data)

    except ValueError:
        # Skip lines with non-numeric values
        continue

except KeyboardInterrupt:
    # Stop gracefully when Ctrl+C is pressed
    print("Data collection stopped.")
    ser.close()

```

5. Now query Firebase to receive all data you have sent so far, process the JSON data array to format each sample as CSV and store in a CSV file.

```

# -----
# 4. Query Firebase
# -----
data_snapshot = ref.get()

# Convert to DataFrame
records = []

```

```

if data_snapshot:
    for key, value in data_snapshot.items():
        records.append(value)

df = pd.DataFrame(records)

# -----
# 5. Clean Data
# -----
# Remove any rows with missing or non-numeric values
df = df.dropna()
df = df[pd.to_numeric(df['x'], errors='coerce').notnull()]
df = df[pd.to_numeric(df['y'], errors='coerce').notnull()]
df = df[pd.to_numeric(df['z'], errors='coerce').notnull()]

# Save to CSV
csv_filename = 'gyroscope_data.csv'
df.to_csv(csv_filename, index=False)
print(f"Data saved to {csv_filename}")

```

6. Try to clean data variables in the CSV file which includes removing any non-number or empty fields, if exists. This step can be a manual process, or you can use Python tools such as Numpy or Pandas, if you want to. Justify your actions.

```

# -----
# 4. Query Firebase
# -----
data_snapshot = ref.get()

# Convert to DataFrame
records = []
if data_snapshot:
    for key, value in data_snapshot.items():
        records.append(value)

df = pd.DataFrame(records)

# -----
# 5. Clean Data
# -----
# Remove any rows with missing or non-numeric values

```

```
df = df.dropna()
df = df[pd.to_numeric(df['x'], errors='coerce').notnull()]
df = df[pd.to_numeric(df['y'], errors='coerce').notnull()]
df = df[pd.to_numeric(df['z'], errors='coerce').notnull()]
```

```
# Save to CSV
csv_filename = 'gyroscope_data.csv'
df.to_csv(csv_filename, index=False)
print(f"Data saved to {csv_filename}")
```

7. Read the CSV file, plot x, y, z variables separately as 3 line-graphs and a graph with all 3 variables in a single graph.

```
# -----
# 6. Plot Data
# -----
plt.figure(figsize=(15,5))

# Plot X, Y, Z separately
plt.subplot(1, 3, 1)
plt.plot(df['timestamp'], df['x'], color='r')
plt.title('Gyroscope X')
plt.xlabel('Timestamp (ms)')
plt.ylabel('X angular velocity')

plt.subplot(1, 3, 2)
plt.plot(df['timestamp'], df['y'], color='g')
plt.title('Gyroscope Y')
plt.xlabel('Timestamp (ms)')
plt.ylabel('Y angular velocity')

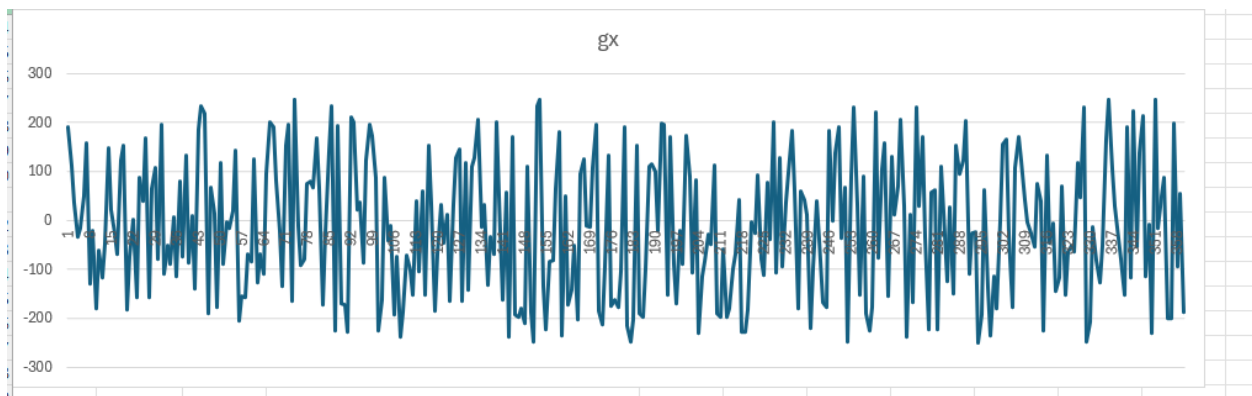
plt.subplot(1, 3, 3)
plt.plot(df['timestamp'], df['z'], color='b')
plt.title('Gyroscope Z')
plt.xlabel('Timestamp (ms)')
plt.ylabel('Z angular velocity')

plt.tight_layout()
plt.show()

# Plot all three on one graph
```

```
plt.figure(figsize=(10,5))
plt.plot(df['timestamp'], df['x'], label='X', color='r')
plt.plot(df['timestamp'], df['y'], label='Y', color='g')
plt.plot(df['timestamp'], df['z'], label='Z', color='b')
plt.title('Gyroscope X, Y, Z')
plt.xlabel('Timestamp (ms)')
plt.ylabel('Angular velocity')
plt.legend()
plt.show()
```

8. Observe the data changing pattern of the single variable plot and combined plot and try to come up with your comments. Following items are subject to observation, but not limited to, -
 - a. Any repeating pattern,
 - b. Up and down trends in data variables, or
 - c. Relative changes of Gyroscope variables x, y or z.
 - d. You should refer to Gyroscope variable semantics in the Arduino document (<https://docs.arduino.cc/tutorials/nano-33-iot/imu-gyroscope>).



GX (Gyroscope X-axis)

a. Repeating Pattern:

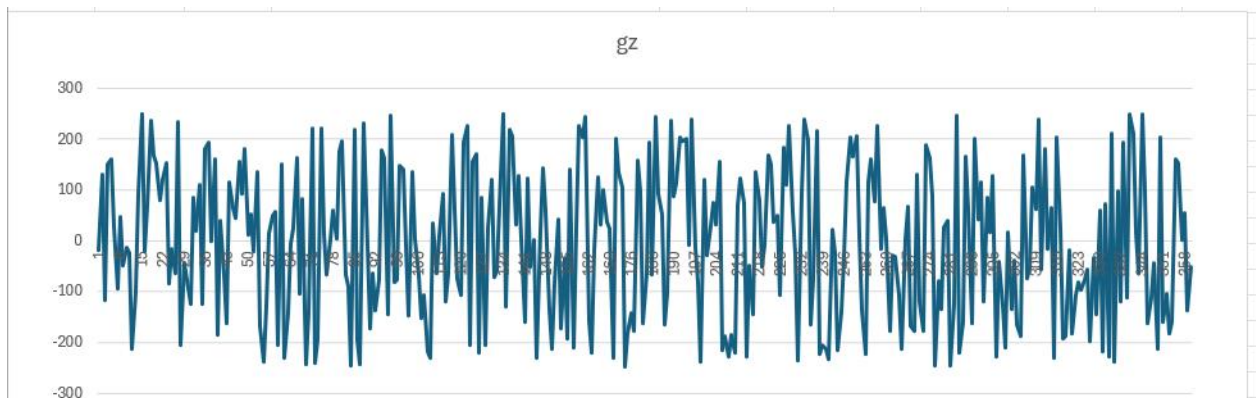
- The GX plot shows **frequent oscillations** with alternating spikes and dips, but no clear periodicity.
- The pattern suggests **intermittent lateral tilting**, possibly from repeated left-right hand movements.

b. Up and Down Trends:

- Sharp upward and downward spikes reaching ± 3000 dps indicate **sudden angular changes**.
- These could correspond to quick rotations or jerks along the X-axis (side-to-side motion).

c. Relative Changes:

- Compared to GY and GZ, GX has the **highest amplitude**, implying that **X-axis rotation was dominant** during data collection.
- This aligns with the Arduino gyroscope semantics: high GX values may indicate strong lateral force or collision from left/right.



GZ (Gyroscope Z-axis)

a. Repeating Pattern:

- GZ shows **lower amplitude** oscillations (± 300 dps), with a more **stable baseline**.

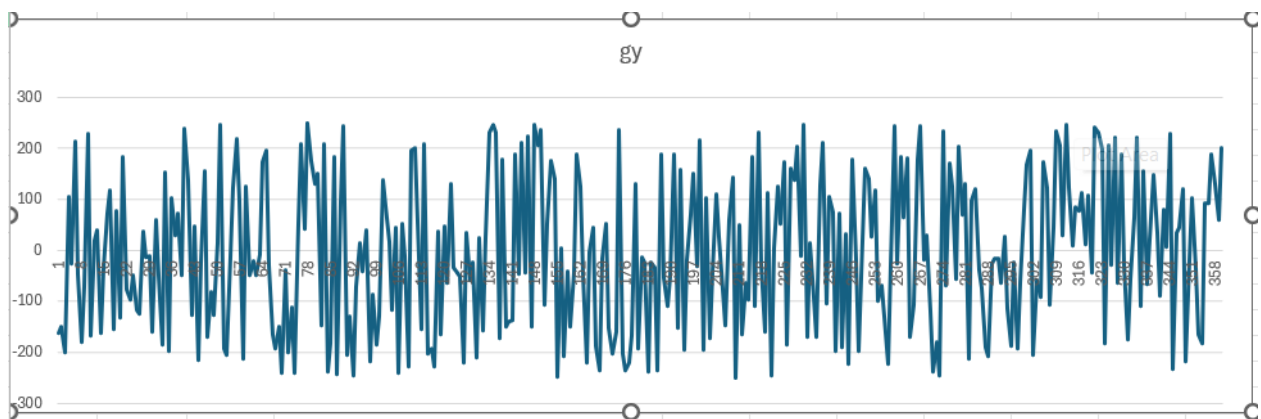
- This suggests **rotational motion around the vertical axis** (yaw), but less intense.

b. Up and Down Trends:

- The fluctuations are tighter and more frequent, possibly indicating **continuous small rotations** or vibrations.
- No major spikes imply **no abrupt spinning** around the Z-axis.

c. Relative Changes:

- GZ is **much calmer** than GX and GY, indicating that **yaw rotation was minimal**.
- This fits with the Arduino gyroscope semantics: Z-axis changes are often subtler unless the board is spun.



GY (Gyroscope Y-axis)

a. Repeating Pattern:

- GY also shows erratic spikes, though slightly less intense than GX.
- There's **no consistent periodicity**, but bursts of activity suggest **forward-backward tilting**.

b. Up and Down Trends:

- Peaks and troughs around ± 3000 dps suggest **rapid pitch movements**—forward or backward tilts.
- These may reflect the board being moved toward or away from the user.

c. Relative Changes:

- GY is **less volatile than GX**, but still shows significant motion.
- According to Arduino's threshold logic, values >30 or

Submission details

Q1. Perform week 5 activities mentioned in the unit site and produce outputs.

Student name: Devindi Chathunika Amarasinghe Rajapaksha Vithanage

Student ID: s225009113

SIT225: Data Capture Technologies

Activity 5.1: Firebase Realtime database

The Firebase Realtime Database is a cloud-hosted NoSQL database that lets you store and sync data between your users in real-time. Data is stored as JSON and synchronized in real-time to every connected client. In this activity, you will set up and perform operations such as queries and updates on the database using Python programming language.

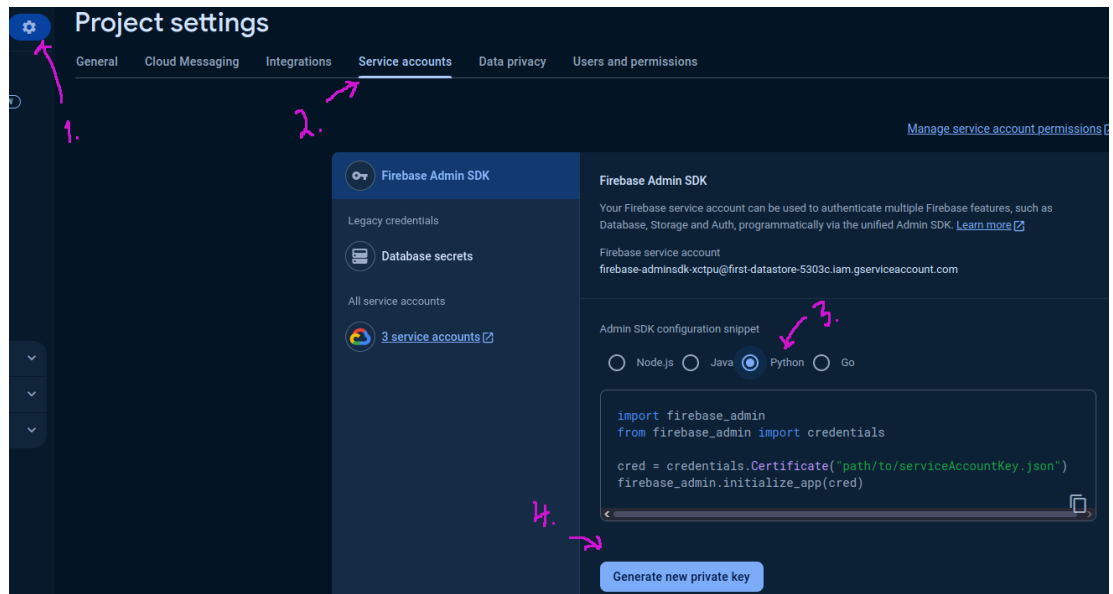
Hardware Required

No hardware is required.

Software Required

Steps

Step	Action
1	Create an Account: First, you will need to create an account in the Firebase console, follow instructions in the official Firebase document (https://firebase.google.com/docs/database/rest/start).
2	Create a Database: Follow the above Firebase document to create a database. When you click on Create Database, you have to specify the location of the database and the security rules. Two rules are available – locked mode and test mode; since we will be using the database for reading, writing, and editing, we choose test mode.
3	Setup Python library for Firebase access: We will be using Admin Database API, which is available in <i>firebase_admin</i> library. Use the below command in the command line to install. You can follow a Firebase tutorial here (https://www.freecodecamp.org/news/how-to-get-started-with-firebase-using-python). \$ pip install firebase_admin Firestore will allow access to Firestore server APIs from Google Service Accounts. To authenticate the Service Account, we require a private key in JSON format. To generate the key, go to project settings, click Generate new private key, download the file, and place it in your current folder where you will create your Python script.



4

Connect to Firebase using Python version of Admin Database API:

A credential object needs to be created to initialise the Python library which can be done using the Python code below. Python notebook can be downloaded here

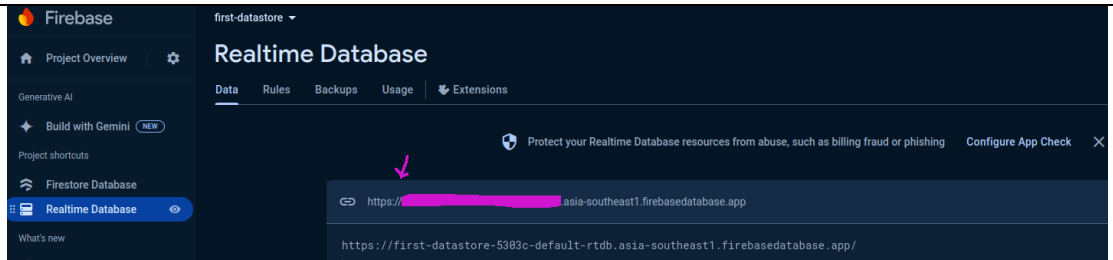
(https://github.com/deakin-deep-dreamer/sit225/blob/main/week_5/firebase_explore.ipynb).

```

1 import firebase_admin
2
3 databaseURL = 'https://XXX.firebaseioapp/'
4 cred_obj = firebase_admin.credentials.Certificate(
5     'first-datastore-5303c-firebase-adminsdk-xctpu-c9902044ac.json'
6 )
7 default_app = firebase_admin.initialize_app(cred_obj, {
8     'databaseURL': databaseURL
9 })

```

The databaseURL is a web address to reach your Firebase database that you have created in step 2. This URL can be found in the Data tab of Realtime Database.



If you compile the code snippet above, it should do with no error.

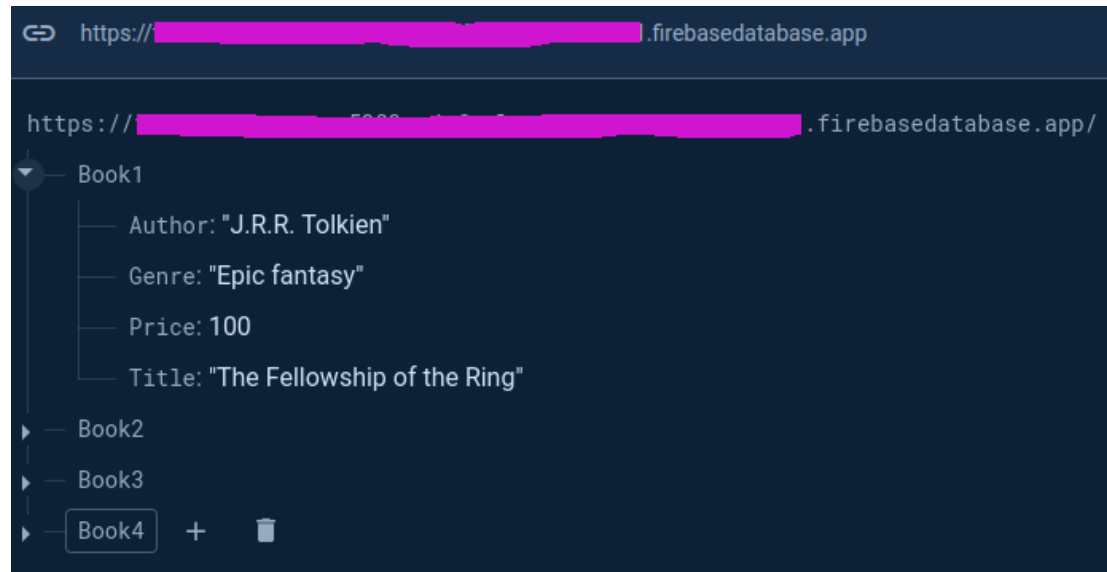
5 Write to database Using the set() Function:

We set the reference to the root of the database (or we could also set it to a key value or child key value). Data needs to be in JSON format as below.

```
1 from firebase_admin import db
2
3 # A reference point is always needed to be set
4 # before any operation is carried out on a database.
5 #
6 ref = db.reference("/")
7
8 # JSON format data (key/value pair)
9 data = { # Outer {} contains inner data structure
10     "Book1":
11         {
12             "Title": "The Fellowship of the Ring",
13             "Author": "J.R.R. Tolkien",
14             "Genre": "Epic fantasy",
15             "Price": 100
16         },
17     "Book2":
18         {
19             "Title": "The Two Towers",
20             "Author": "J.R.R. Tolkien",
21             "Genre": "Epic fantasy",
22             "Price": 100
23         },
24     "Book3":
25         {
26             "Title": "The Return of the King",
27             "Author": "J.R.R. Tolkien",
28             "Genre": "Epic fantasy",
29             "Price": 100
30         },
31     "Book4":
32         {
33             "Title": "Brida",
34             "Author": "Paulo Coelho",
35             "Genre": "Fiction",
36             "Price": 100
37         }
38 }
39
40 # JSON format data is set (overwritten) to the reference
41 # point set at /, which is the root node.
42 #
43 ref.set(data)
```

A reference point always needed to be set where the data read/write will take place. In the code above, the reference point is set at the root of the NoSQL Document, where consider the database is a JSON tree and / is the root node of the tree). The set() function writes (overwrites) data at the set reference point.

You can visualise the data in the Firebase console as below -



6 **Read data using get() function:**

Data can be read using get() function on the reference set beforehand, as shown below.

	<pre> 1 ref = db.reference("/") # set ref point 2 3 # query all data under the ref 4 books = ref.get() 5 print(books) 6 print(type(books)) 7 8 # print each item separately 9 for key, value in books.items(): 10 print(f"{key}: {value}") 11 12 13 # Query /Book1 14 ref = db.reference("/Book1") 15 books = ref.get() 16 print(books) </pre> ✓ 0.3s <pre> {'Book1': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Hobbit'}} <class 'dict'> Book1: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Hobbit'} Book2: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Hobbit'} Book3: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Hobbit'} Book4: {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'} {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Hobbit'} </pre> Consider the reference set in line 1 and the output compared to the reference set at line 14 and the bottom output line to understand the use of db.reference() and ref.get().
7	Write to database Using the push() Function: The push() function saves data under a <i>unique system generated key</i>. This is different than set() where you set the keys such as Book1, Book2, Book3 and Book4 under which the content (author, genre, price and title) appears. Let's try to push the same data in the root reference. Note that since we already has data under root / symbol, setting (or pushing) in the same reference point will eventually rewrite the original data.

```

1 # Write using push() function
2 # Note that a set() is called on top of push()
3 #
4 ref = db.reference("/")
5 ref.set({
6     "Books":
7     {
8         "Best_Sellers": -1
9     }
10 })
11
12 ref = db.reference("/Books/Best_Sellers")
13
14 for key, value in data.items():
15     ref.push().set(value)

```

✓ 2.0s

The output will reset the previous data set in / node. The current data is shown below.

```

▼ — Books
  ▼ — Best_Sellers
    ▼ — -0-iqpiYlui92UKRmctM
        — Author: "J.R.R. Tolkien"
        — Genre: "Epic fantasy"
        — Price: 100
        — Title: "The Fellowship of the Ring"
    ▶ — -0-iqpnK8M8wjLiw2PTX
    ▶ — -0-iqptGIKG7WuxHdGsQ
    ▶ — -0-iqpz_nsDjhwMzLmIw

```

As you can see, under /Books/Best_Sellers there are 4 nodes where the node head (or node ID) is a randomly generated key which is due to the use of push() function. When data key does not matter, the use of push() function is desirable.

8

Update data:

Let's say the price of the books by J. R. R. Tolkien is reduced to 80 units to offer a discount. The first 3 books are written by this author, and we want to apply for a discount on all of them.

```
1 # Update data
2 #
3 # Requirement: The price of the books by
4 # J. R. R. Tolkien is reduced to 80 units to
5 # offer a discount.
6 #
7 ref = db.reference("/Books/Best_Sellers/")
8 best_sellers = ref.get()
9 print(best_sellers)
10 for key, value in best_sellers.items():
11     if(value["Author"] == "J.R.R. Tolkien"):
12         value["Price"] = 90
13         ref.child(key).update({"Price":80})
```

✓ 0.9s

As you can see, the author name is compared and the new price is set in the best_sellers dictionary and finally, an update() function is called on the ref, however, the current ref is a '/Books/Best_Sellers/', so we need to locate the child under the ref node, so ref.child(key) is used in line 13. The output is shown below with a discounted price.

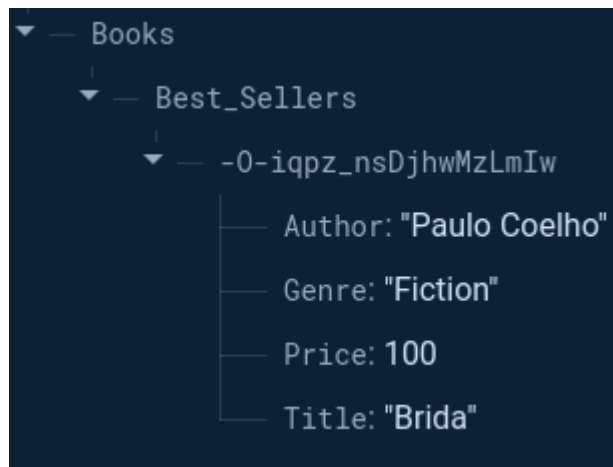
		<pre> ▼ — Best_Sellers ▼ — -0-iqpiYlui92UKRmctM — Author: "J.R.R. Tolkien" — Genre: "Epic fantasy" — Price: 80 — Title: "The Fellowship of the Ring" ▼ — -0-iqpnK8M8wjLiw2PTX — Author: "J.R.R. Tolkien" — Genre: "Epic fantasy" — Price: 80 — Title: "The Two Towers" ▼ — -0-iqptGIKG7WuxHdGsQ — Author: "J.R.R. Tolkien" — Genre: "Epic fantasy" — Price: 80 — Title: "The Return of the King" ▼ — -0-iqpz_nsDjhwMzLmIw — Author: "Paulo Coelho" </pre>
9	Delete data: Let's delete all bestseller books with J.R.R. Tolkien as the author. You can locate the node using <code>db.reference()</code> (line 4) and then locate specific record (for loop in line 6) and calling <code>set()</code> with empty data <code>{}</code> as a parameter, such as <code>set({})</code>. The particular child under the ref needs to be located first by using <code>ref.child(key)</code>, otherwise, the ref node will be removed – BE CAREFUL.	


```

1 # Let's delete all best seller books
2 # with J.R.R. Tolkien as the author.
3 #
4 ref = db.reference("/Books/Best_Sellers")
5
6 for key, value in best_sellers.items():
7     if(value["Author"] == "J.R.R. Tolkien"):
8         ref.child(key).set({})

```

This keeps only the other author data, as shown below.



If ref.child() not used, as shown the code below, all data will be removed.

```

1 ref = db.reference("/Books/Best_Sellers")
2 ref.set({})

```

Now in Firebase console you will see no data exists.

10 **Question:** Run all the cells in the Notebook you have downloaded in Step 4, fill in the student information at the top cell of the Notebook. Convert the Notebook to PDF and merge with this activity sheet PDF.

Answer: Convert the Notebook to PDF and merge with this activity sheet PDF.

10 **Question:** Create a sensor data structure for DHT22 sensor which contains attributes such as sensor_name, timestamp, temperature and humidity. Remember there will be other sensors with different sensor variables such as DHT22 has 2 variables, accelerometer sensor has 3. For each such sensor,

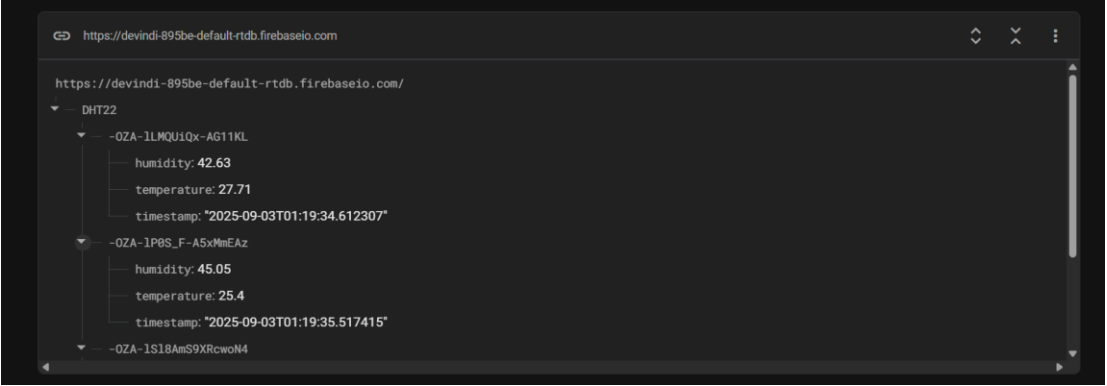
you will need to gather data over time. Discuss how you are going to handle multiple data values in JSON format? Justify your design.

Answer:

```
sensorData={
  "sensors": [
    {
      "sensor_name": "DHT22",
      "readings": [
        {
          "timestamp": "2025-09-02T10:00:00",
          "temperature": 25.3,
          "humidity": 60
        },
        {
          "timestamp": "2025-09-02T10:05:00",
          "temperature": 25.5,
          "humidity": 61
        }
      ]
    },
    {
      "sensor_name": "Accelerometer",
      "readings": [
        {
          "timestamp": "2025-09-02T10:00:00",
          "x": 0.12,
          "y": 0.01,
          "z": 0.98
        },
        {
          "timestamp": "2025-09-02T10:05:00",
          "x": 0.15,
          "y": 0.03,
          "z": 0.97
        }
      ]
    }
  ]
}
```

The proposed sensor data structure is highly flexible and extensible. New types of sensors can be added easily without modifying the existing structure because each sensor has its own unique *sensor_name* and a separate *readings* list. Each reading is stored as a dictionary containing a *timestamp* and the sensor-specific variables, which allows the data to be collected and organized over time. Using a list to store

	multiple readings ensures that historical data is preserved and can be accessed for analysis or visualization. Additionally, the structure is clear and organized, with each sensor's data kept separate, making it easy to process programmatically. It is also JSON-friendly, enabling seamless storage in databases or files and allowing easy integration with systems such as Firebase or other data platforms. Overall, this design provides scalability, clarity, and efficient handling of multiple sensor types and their time-series data.
11	Question: Generate some random data for DHT22 sensor, insert data to database, query all data and screenshot the output here. Answer: <pre> import random from datetime import datetime from firebase_admin import db # Reference to the DHT22 node in your Firebase database ref = db.reference("/DHT22") # Function to generate random DHT22 data def generate_dht22_data(): return { "timestamp": datetime.now().isoformat(), "temperature": round(random.uniform(20.0, 30.0), 2), # Random temp 20-30°C "humidity": round(random.uniform(40.0, 70.0), 2) # Random humidity 40-70% } # Insert 5 random readings into the database for _ in range(5): reading = generate_dht22_data() ref.push(reading) # Push creates a unique key for each reading # Query all data from the DHT22 node all_data = ref.get() # Print all data print("All DHT22 Sensor Data:") for key, value in all_data.items(): print(f"{key}: {value}") </pre> All DHT22 Sensor Data: <pre> -OZA-1LMQuiQx-AG11KL: {'humidity': 42.63, 'temperature': 27.71, 'timestamp': '2025-09-03T01:19:34.612307'} -OZA-1P0S_F-A5xMmEAz: {'humidity': 45.05, 'temperature': 25.4, 'timestamp': '2025-09-03T01:19:35.517415'} -OZA-1S18AmS9XRcwoN4: {'humidity': 47.5, 'temperature': 25.48, 'timestamp': '2025-09-03T01:19:35.760350'} -OZA-1WZD_ezCHyY-Pno: {'humidity': 69.09, 'temperature': 22.03, 'timestamp': '2025-09-03T01:19:36.000975'} -OZA-1_OZpGpGtPoc3il: {'humidity': 67.05, 'temperature': 28.8, 'timestamp': '2025-09-03T01:19:36.247323'} </pre>

	
12	Question: Generate some random data for the SR04 Ultrasonic sensor, insert data to database, query all data and screenshot the output here. Answer:

```

import random
from datetime import datetime
from firebase_admin import db

# Reference to the SR04 node
ref = db.reference("/SR04")

# Function to generate random SR04 data (distance in cm)
def generate_sr04_data():
    return {
        "timestamp": datetime.now().isoformat(),
        "distance_cm": round(random.uniform(2.0, 400.0), 2) # SR04 range: 2cm - 400cm
    }

# Insert 5 random readings into the database
for _ in range(5):
    reading = generate_sr04_data()
    ref.push(reading) # push creates a unique key for each reading

# Query all data from the SR04 node
all_data = ref.get()

# Print all data
print("All SR04 Sensor Data:")
for key, value in all_data.items():
    print(f"{key}: {value}")

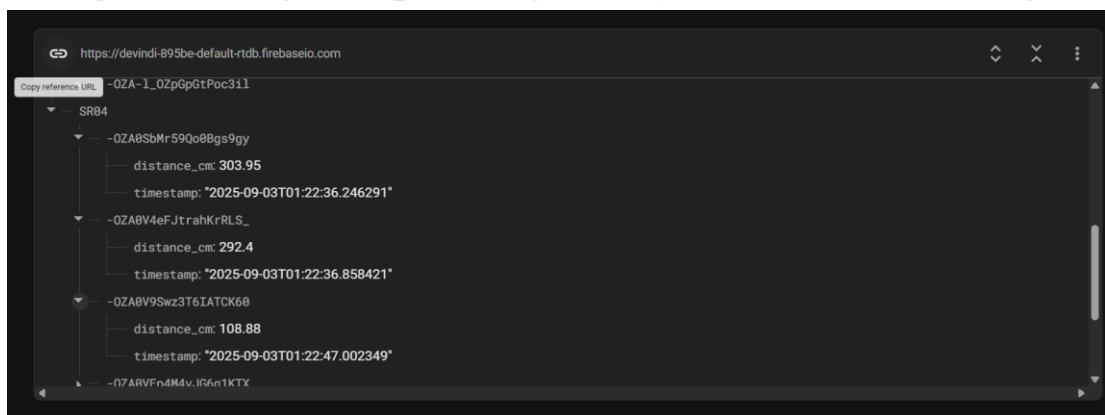
```

All SR04 Sensor Data:

```

-OZA0SbMr59Qo0Bgs9gy: {'distance_cm': 303.95, 'timestamp': '2025-09-03T01:22:36.246291'}
-OZA0V4eFJtrahKrRLS_: {'distance_cm': 292.4, 'timestamp': '2025-09-03T01:22:36.858421'}
-OZA0V9Swz3T6IATCK60: {'distance_cm': 108.88, 'timestamp': '2025-09-03T01:22:47.002349'}
-OZA0VEp4M4vJG6g1KTX: {'distance_cm': 260.41, 'timestamp': '2025-09-03T01:22:47.307943'}
-OZA0VJkZcykOSIq6BvA: {'distance_cm': 174.96, 'timestamp': '2025-09-03T01:22:47.649511'}
-OZA0XRwp3NzQVJyt_rn: {'distance_cm': 216.2, 'timestamp': '2025-09-03T01:22:56.375033'}
-OZA0XWcrnZBvvivjkhy: {'distance_cm': 254.6, 'timestamp': '2025-09-03T01:22:56.679739'}
-OZA0XaLckpJiYoDQ02: {'distance_cm': 71.62, 'timestamp': '2025-09-03T01:22:56.983011'}
-OZA0Xf6Grmcu1w98QJ: {'distance_cm': 214.37, 'timestamp': '2025-09-03T01:22:57.282450'}
-OZA0Xjn-xIoJzUtl03: {'distance_cm': 119.03, 'timestamp': '2025-09-03T01:22:57.589014'}

```



13

Question: Firebase Realtime database generates events on data operations. You can refer to section 'Handling Realtime Database events' in the document (<https://firebase.google.com/docs/functions/database-events?gen=2nd>). Discuss in the active learning session and summarise the idea of database events and how it is handled using Python SDK.

Note that these events are useful when your sensors (from Arduino script) store data directly to Firebase Realtime database and you would like to track data update actions from a central Python application such as a monitoring dashboard.

Answer:

What Are Database Events?

Firebase Realtime Database generates **event triggers** when data changes occur. These events allow applications to **react automatically** to updates, without polling.

Event Name	Description
onValueCreated()	Triggered when new data is added
onValueUpdated()	Triggered when existing data is modified
onValueDeleted()	Triggered when data is removed
onValueWritten()	Triggered on any create, update, or delete

Handling Events with Python SDK

Using the **Firebase Admin SDK for Python**, you can define functions that respond to these events.

Key Concepts:

- **Event snapshot:** Contains the data at the time of the event.
- **Before/after comparison:** Available for update events.
- **Reference path:** Specifies where in the database the event is triggered.

Example Use Case:

If a sensor like **DHT22** or **SR04** stores readings in Firebase, a Python app can:

- Track new readings in real-time
- Log data for analysis
- Trigger alerts if thresholds are exceeded

Activity 5.2: Data wrangling

Data wrangling is the process of converting raw data into a usable form. The process includes collecting, processing, analyzing, and tidying the raw data so that it can be easily read and analyzed. In this activity, you will use the common library in python, "pandas".

Hardware Required

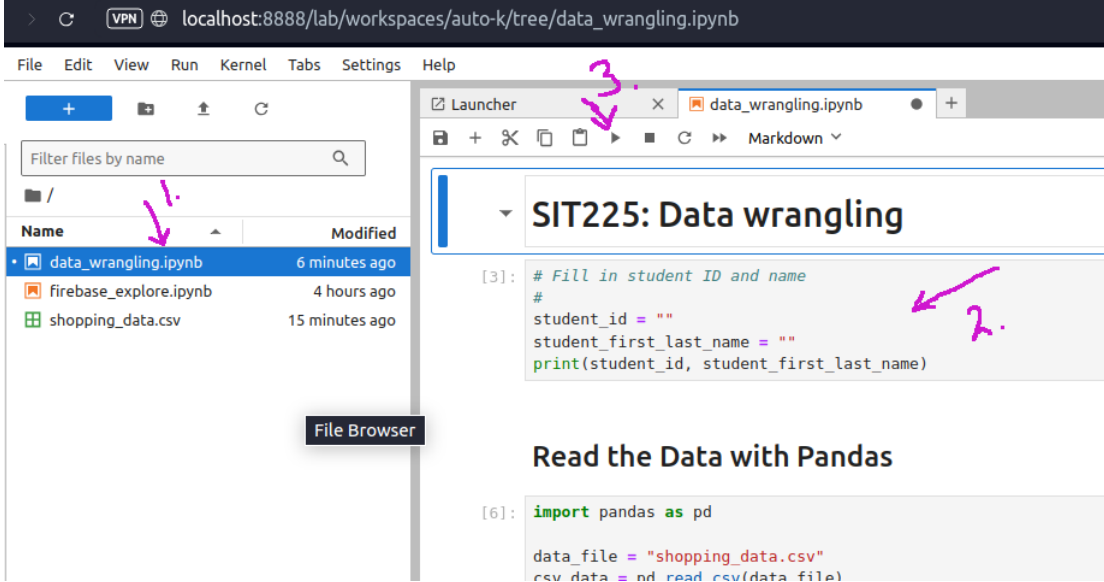
No hardware is required.

Software Required

Python 3
Pandas Python library

Steps

Step	Action
1	Install Pandas using the command below. Most likely you already have Pandas installed if you have installed Python using Anaconda distribution (https://www.anaconda.com/download). <pre>\$ pip install pandas</pre> A Python notebook is shared in the GitHub link (https://github.com/deakin-deep-dreamer/sit225/tree/main/week_5). There will be a <code>data_wrangling.ipynb</code>, <code>shopping_data.csv</code> and <code>shopping_data_missingvalue.csv</code> files among others. Download the <code>week_5</code> folder in your computer, open a command prompt in that folder, and write the command below in the command line: <pre>\$ jupyter lab</pre> This will open Python Jupyter Notebook where in the left panel you can see the files (labeled as 1 in figure).

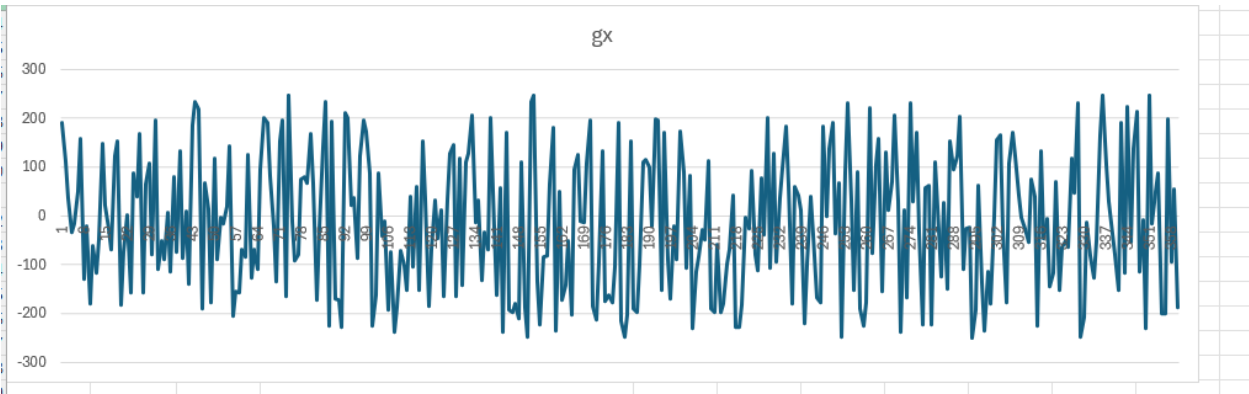
	 Each cell contains Python code (labeled as 2 in figure), you can run a cell by clicking on the cell, so the cursor appears in that cell and then click on the play button at the top of the panel (labeled as 3 in the figure).
2	Question: Run each cell to produce output. Follow instructions in the notebook to complete codes in some of the cells. Convert the notebook to PDF from menu File > Save and Export Notebook As > PDF. Convert this activity sheet to PDF and merge with the notebook PDF. Answer: There is no answer to write here. You have to answer in the Jupyter Notebook.
3	Question: Once you went through the cells in the Notebook, you now have a basic understanding of data wrangling. Pandas are a powerful tool and can be used for reading CSV data. Can you use Pandas in reading sensor CSV data that you generated earlier? Describe if any modification you think necessary? Answer: Yes, Pandas is highly suitable for reading sensor-generated CSV data. Using <code>pd.read_csv()</code>, we can efficiently load and inspect sensor readings such as temperature, humidity, or distance measurements from devices like DHT22 or SR04. However, some modifications may be necessary depending on the structure and quality of the data:

	• Timestamp Parsing: If the sensor data includes timestamps, use <code>parse_dates=['timestamp']</code> to ensure proper datetime formatting. • Missing Values: Sensors may occasionally fail to record data, resulting in missing entries. These should be handled using imputation or filtering. • Data Types: Ensure numeric columns are correctly typed using <code>df.astype()</code> if needed, especially if readings are stored as strings. • Outlier Detection: Sensor glitches may produce extreme values. Use statistical methods or domain thresholds to identify and clean these. • Indexing: Setting the timestamp as an index (<code>df.set_index('timestamp')</code>) can help with time-series analysis and plotting. Overall, Pandas provides a flexible and powerful framework for preprocessing sensor data before feeding it into machine learning models or dashboards.															
4	Question: What do you understand of the Notebook section called Handling Missing Value? Discuss in group and briefly summarise different missing value imputation methods and their applicability on different data conditions. Answer: The "Handling Missing Value" section introduces techniques for dealing with incomplete data, which is common in real-world datasets including sensor logs. Missing values can distort analysis and model performance if not addressed properly. <table><tr><th>Method</th><th>Description</th><th>Best Used When</th></tr><tr><td><code>Dropna()</code></td><td>Removes rows or columns with missing values</td><td>Data loss is minimal and missingness is random</td></tr><tr><td><code>Fillna(value)</code></td><td>Replaces missing values with a fixed value (e.g., 0 or mean)</td><td>Simple datasets or when domain value is known</td></tr><tr><td><code>fillna(method=</code></td><td>Forward fill: propagates last valid value forward</td><td>Time-series data with gradual changes</td></tr><tr><td><code>fillna(method=</code></td><td>Backward fill: uses next valid value</td><td>Time-series with consistent trends</td></tr></table>	Method	Description	Best Used When	<code>Dropna()</code>	Removes rows or columns with missing values	Data loss is minimal and missingness is random	<code>Fillna(value)</code>	Replaces missing values with a fixed value (e.g., 0 or mean)	Simple datasets or when domain value is known	<code>fillna(method=</code>	Forward fill: propagates last valid value forward	Time-series data with gradual changes	<code>fillna(method=</code>	Backward fill: uses next valid value	Time-series with consistent trends
Method	Description	Best Used When														
<code>Dropna()</code>	Removes rows or columns with missing values	Data loss is minimal and missingness is random														
<code>Fillna(value)</code>	Replaces missing values with a fixed value (e.g., 0 or mean)	Simple datasets or when domain value is known														
<code>fillna(method=</code>	Forward fill: propagates last valid value forward	Time-series data with gradual changes														
<code>fillna(method=</code>	Backward fill: uses next valid value	Time-series with consistent trends														

	mean()/median()	Replaces missing values with statistical aggregates	Numeric data with low variance
	Interpolation	Estimates missing values using linear or polynomial interpolation	Continuous data with smooth transitions

PDF Will attached below.

Q2. State the hypothesis you can think of out of your data. Show the graph created from the sensor data, analyse it and describe if there are any interesting patterns you can observe. Justify if your hypothesis holds, at what level; if not, then what might be the reason?



GX (Gyroscope X-axis)

a. Repeating Pattern:

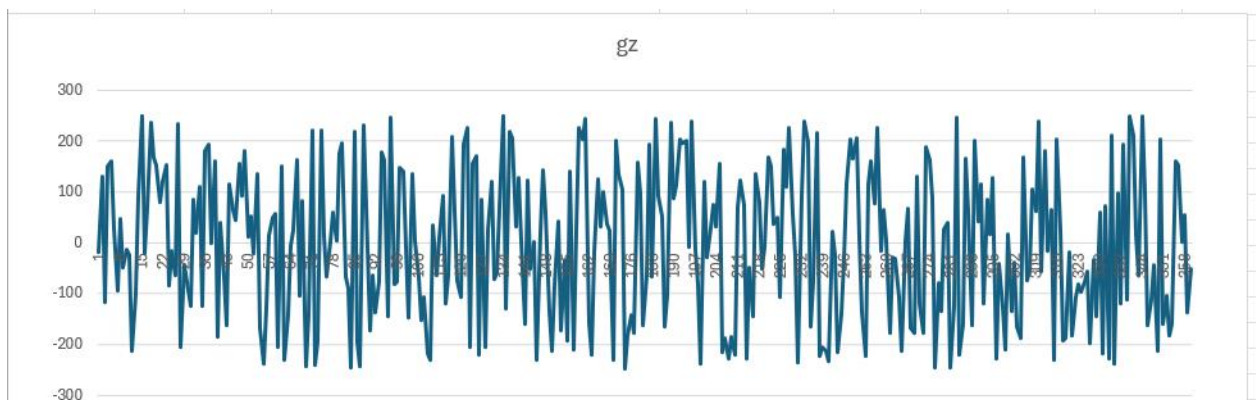
- The GX plot shows **frequent oscillations** with alternating spikes and dips, but no clear periodicity.
- The pattern suggests **intermittent lateral tilting**, possibly from repeated left-right hand movements.

b. Up and Down Trends:

- Sharp upward and downward spikes reaching ± 3000 dps indicate **sudden angular changes**.
- These could correspond to quick rotations or jerks along the X-axis (side-to-side motion).

c. Relative Changes:

- Compared to GY and GZ, GX has the **highest amplitude**, implying that **X-axis rotation was dominant** during data collection.
- This aligns with the Arduino gyroscope semantics: high GX values may indicate strong lateral force or collision from left/right.



GZ (Gyroscope Z-axis)

a. Repeating Pattern:

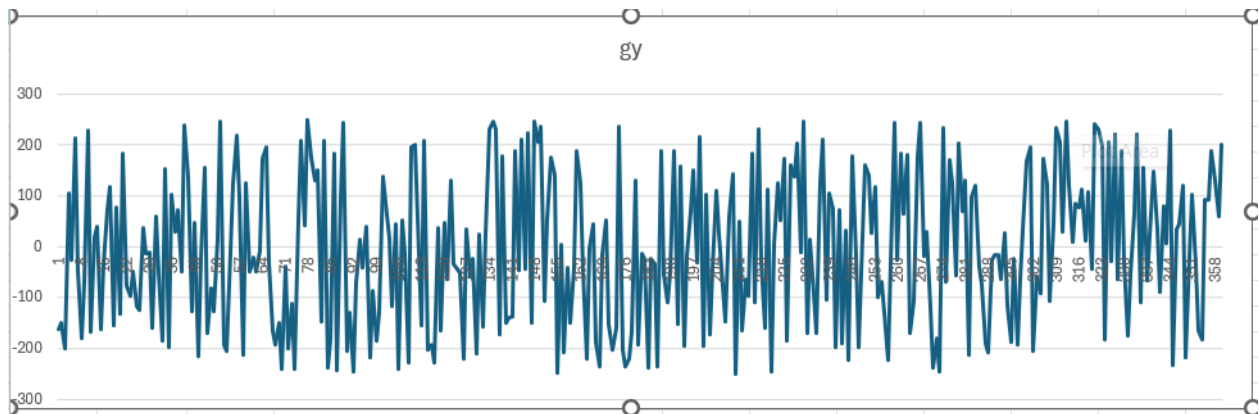
- GZ shows **lower amplitude** oscillations (± 300 dps), with a more **stable baseline**.
- This suggests **rotational motion around the vertical axis** (yaw), but less intense.

b. Up and Down Trends:

- The fluctuations are tighter and more frequent, possibly indicating **continuous small rotations** or vibrations.
- No major spikes imply **no abrupt spinning** around the Z-axis.

c. Relative Changes:

- GZ is **much calmer** than GX and GY, indicating that **yaw rotation was minimal**.
- This fits with the Arduino gyroscope semantics: Z-axis changes are often subtler unless the board is spun.



GY (Gyroscope Y-axis)

a. Repeating Pattern:

- GY also shows erratic spikes, though slightly less intense than GX.
- There's **no consistent periodicity**, but bursts of activity suggest **forward-backward tilting**.

b. Up and Down Trends:

- Peaks and troughs around ± 3000 dps suggest **rapid pitch movements**—forward or backward tilts.
- These may reflect the board being moved toward or away from the user.

c. Relative Changes:

- GY is **less volatile than GX**, but still shows significant motion.
- According to Arduino's threshold logic, values >30 or

Q3. Paste Python and Arduino sketch and explain program steps.

```

1 #include <Arduino_LSM6DS3.h> // Library for IMU sensors
2
3 void setup() {
4   Serial.begin(9600); // Start Serial communication
5   while (!Serial); // Wait for Serial port to connect (needed for Nano 33 IoT)
6
7   if (!IMU.begin()) { // Initialize IMU
8     Serial.println("Failed to initialize IMU");
9     while (1);
10  }
11
12  Serial.println("timestamp_ms,x,y,z"); // CSV header for easier parsing
13 }
14
15 void loop() {
16   float gx, gy, gz;
17
18   // Check if gyroscope data is available
19   if (IMU.gyroscopeAvailable()) {
20     IMU.readGyroscope(gx, gy, gz); // Read x, y, z angular velocities
21
22     unsigned long timestamp = millis(); // Current timestamp in milliseconds
23
24     // Send data as CSV: timestamp,x,y,z
25     Serial.print(timestamp);
26     Serial.print(",");
27     Serial.print(gx);
28     Serial.print(",");
29     Serial.print(gy);
30     Serial.print(",");
31     Serial.println(gz);
32   }
33
34   delay(100); // Sample every 100ms + 10Hz data rate (adjust as needed)
35 }
36

```

Output

Ln 36, Col 1 Arduino NANO 33 IoT on COM4

The Arduino code begins by including the `Arduino_LSM6DS3` library, which allows communication with the gyroscope and accelerometer on the Nano 33 IoT, giving access to angular velocities along the x, y, and z axes. In the `setup()` function, Serial communication is initialized at 9600 baud to transmit data to a computer, and the code waits until the Serial port is ready. The gyroscope is

then initialized, and if it fails, an error message is printed and the program halts. A CSV header (timestamp_ms,x,y,z) is printed to make parsing the data easier later. In the loop() function, three floating-point variables are declared to store x, y, and z angular velocities. The code checks if new gyroscope data is available and, if so, reads the values into these variables. A timestamp in milliseconds is recorded to indicate when the data was captured. The timestamp and x, y, z readings are sent over Serial in CSV format for easy collection. Finally, a delay of 100 milliseconds controls the sampling frequency, resulting in a 10 Hz rate, which balances sufficient motion detail with manageable data transmission.

```
Receive_Data_and_Upload_to_Firebase.py - C:/Users/Devindi Amarasinghe/Receive_Data_and_Upload_to_Firebase.py (3.12.4)
File Edit Format Run Options Window Help

import serial
import time
from datetime import datetime
import firebase_admin
from firebase_admin import credentials, db
import pandas as pd
import matplotlib.pyplot as plt

# -----
# 1. Firebase Setup
# -----
cred = credentials.Certificate("C:/Users/Devindi Amarasinghe/Downloads/devindi-895be-firebase-adminsdk-fb5vc-4b842e80e5.json")
firebase_admin.initialize_app(cred, {
    "databaseURL": "https://devindi-895be-default-rtdb.firebaseio.com/"
})

# Firebase reference
ref = db.reference('gyroscope_data')

# -----
# 2. Serial Setup
# -----
ser = serial.Serial('COM3', 9600)
time.sleep(2) # Wait for Serial to initialize

# -----
# 3. Receive and Upload Data
# -----
try:
    while True:
        line = ser.readline().decode('utf-8').strip() # Read line from Serial
        if line.startswith("timestamp_ms"): # Skip header
            continue

        # Split CSV data into components
        parts = line.split(',')
        if len(parts) != 4:
            continue # Skip invalid lines

        timestamp_ms, x, y, z = parts

        try:
            # Prepare JSON object
            data = {
                'timestamp': int(timestamp_ms),
                'x': float(x),
                'y': float(y),

```

11°C Light rain 11:54 PM 9/3/2025

```
Receive_Data_and_Upload_to_Firebase.py - C:/Users/Devindi Amarasinghe/Receive_Data_and_Upload_to_Firebase.py (3.12.4)
File Edit Format Run Options Window Help
})

# Firebase reference
ref = db.reference('gyroscope_data')

# -----
# 2. Serial Setup
# -----
ser = serial.Serial('COM3', 9600)
time.sleep(2) # Wait for Serial to initialize

# -----
# 3. Receive and Upload Data
# -----
try:
    while True:
        line = ser.readline().decode('utf-8').strip() # Read line from Serial
        if line.startswith("timestamp_ms"): # Skip header
            continue

        # Split CSV data into components
        parts = line.split(',')
        if len(parts) != 4:
            continue # Skip invalid lines

        timestamp_ms, x, y, z = parts

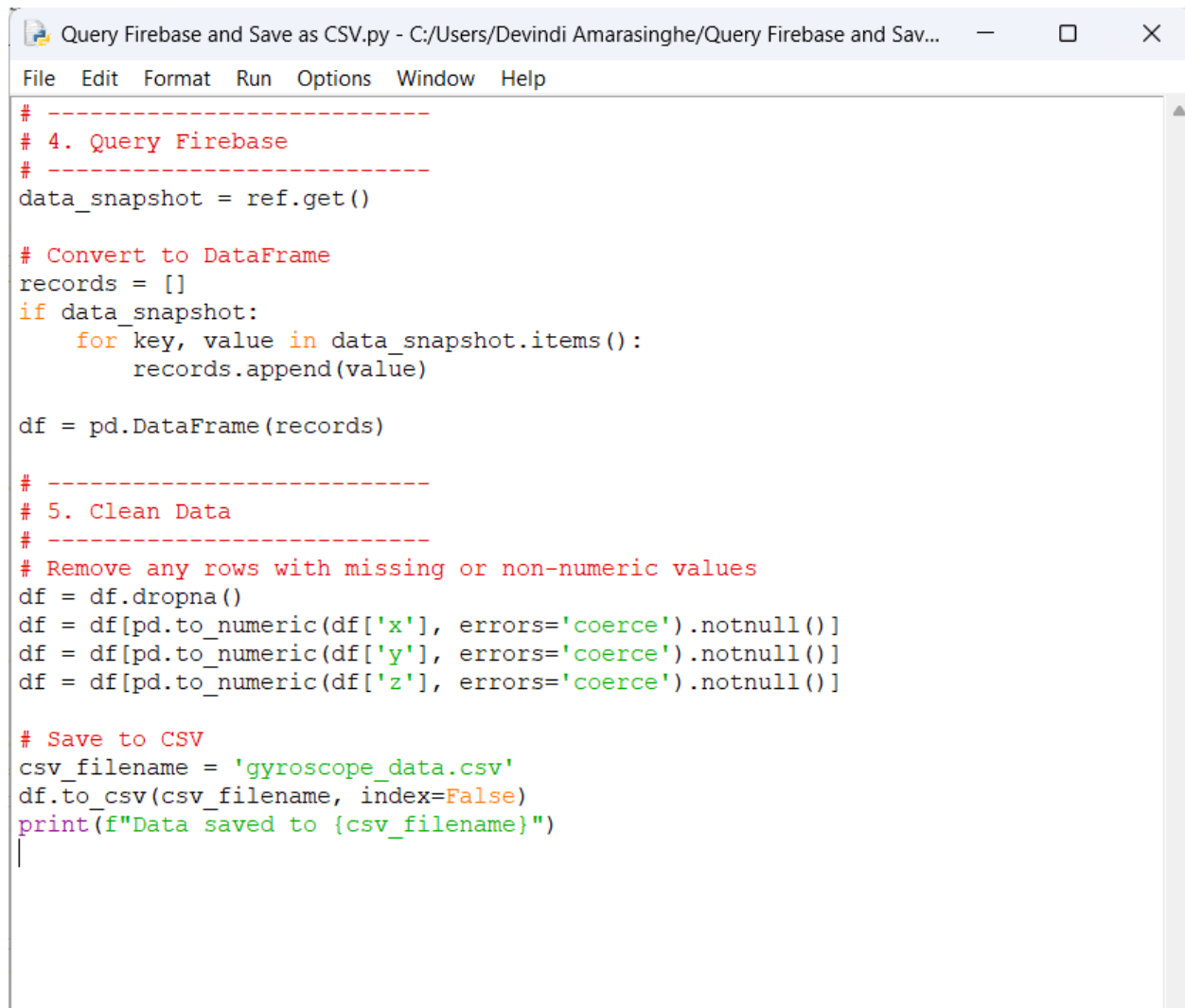
        try:
            # Prepare JSON object
            data = {
                'timestamp': int(timestamp_ms),
                'x': float(x),
                'y': float(y),
                'z': float(z)
            }

            # Push to Firebase
            ref.push(data)

        except ValueError:
            # Skip lines with non-numeric values
            continue

except KeyboardInterrupt:
    print("Data collection stopped.")
    ser.close()
```

This Python code continuously reads gyroscope data from an Arduino via Serial and uploads it to a Firebase real-time database. It begins by importing necessary libraries for Serial communication, time management, Firebase interaction, data handling with Pandas, and plotting with Matplotlib. The Firebase setup authenticates using a service account JSON file and initializes a reference to the `gyroscope_data` node where all readings will be stored. The Serial connection is then established on COM3 at 9600 baud, with a short pause to ensure the port is ready. Inside a try block, the code enters an infinite loop where it reads each line from Serial, skipping the CSV header, and splits the line into timestamp, x, y, and z values. Only lines with exactly four components are processed. The values are converted into a JSON object, with the timestamp as an integer and x, y, z as floats, and this object is pushed to Firebase. Any lines with missing or non-numeric values are skipped to maintain data integrity. The loop continues until interrupted manually, at which point the program prints a stop message and safely closes the Serial port.

A screenshot of a code editor window titled "Query Firebase and Save as CSV.py - C:/Users/Devindi Amarasinghe/Query Firebase and Sav...". The window has a menu bar with "File", "Edit", "Format", "Run", "Options", "Window", and "Help". The code is written in Python and includes several comments in red. The code starts with a comment "# 4. Query Firebase", followed by "data_snapshot = ref.get()". Then, a comment "# Convert to DataFrame" is followed by "records = []". An "if data_snapshot:" block contains a "for key, value in data_snapshot.items():" loop with "records.append(value)". After the loop, "df = pd.DataFrame(records)" is executed. Next, a comment "# 5. Clean Data" is followed by another comment "# Remove any rows with missing or non-numeric values". This is followed by "df = df.dropna()", and three lines of "df = df[pd.to_numeric(df['x'], errors='coerce').notnull()]", "df = df[pd.to_numeric(df['y'], errors='coerce').notnull()]", and "df = df[pd.to_numeric(df['z'], errors='coerce').notnull()]". Finally, a comment "# Save to CSV" is followed by "csv_filename = 'gyroscope_data.csv'", "df.to_csv(csv_filename, index=False)", and "print(f\"Data saved to {csv_filename}\")".

```
# -----  
# 4. Query Firebase  
# -----  
data_snapshot = ref.get()  
  
# Convert to DataFrame  
records = []  
if data_snapshot:  
    for key, value in data_snapshot.items():  
        records.append(value)  
  
df = pd.DataFrame(records)  
  
# -----  
# 5. Clean Data  
# -----  
# Remove any rows with missing or non-numeric values  
df = df.dropna()  
df = df[pd.to_numeric(df['x'], errors='coerce').notnull()]  
df = df[pd.to_numeric(df['y'], errors='coerce').notnull()]  
df = df[pd.to_numeric(df['z'], errors='coerce').notnull()]  
  
# Save to CSV  
csv_filename = 'gyroscope_data.csv'  
df.to_csv(csv_filename, index=False)  
print(f\"Data saved to {csv_filename}\")  
|
```

This part of the Python code retrieves the gyroscope data previously uploaded to Firebase, cleans it, and saves it as a CSV file. First, `ref.get()` fetches all records from the `gyroscope_data` node, returning a dictionary of key-value pairs. Each value, which contains a single data sample, is appended to a list called `records`. This list is then converted into a Pandas `DataFrame` for easier manipulation and analysis. Next, the data is cleaned to ensure quality: `dropna()` removes any rows with missing values, and `pd.to_numeric(..., errors='coerce')` checks that the `x`, `y`, and `z` columns contain only numeric values, discarding any non-numeric entries. Finally, the clean `DataFrame` is saved to a CSV file named `gyroscope_data.csv` without row indices, and a message is printed to confirm that the data has been successfully saved.

Q4. Create a video in Panopto/CloudDeakin showing your program execution, data collection, data upload to Firebase and graph output, share the video link here.

<https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=573df295-0d42-4a6d-b7bf-b34d00f46671&edit=true>

Q5. Create a subdirectory 'week-5' under directory 'SIT225_<YYYY>T2' in your drive where you copy the Python script file, Arduino sketch file, data file and the generated graphs. Commit and push changes to GitHub. Include the link to your repository here with a GitHub page screenshot of weekly folder content. A tutor may try to access your GitHub link, if necessary. Give access to your tutor by adding the tutor's email address as a collaborator of your **private** repository.

https://github.com/Devindi-ops/-SIT225_2025-T2-/tree/week-5

Instructions

Consolidate outputs following the submission details above into a single PDF file.

Submit your work

When you are ready, login to OnTrack and submit your pdf which consolidates all the items mentioned in the submission detail section above. Remember to save and backup your work.

Complete your work

After your submission, your OnTrack reviewer (tutor) will review your submission and give you feedback in about 5 business days. Your reviewer may further ask you some questions on the weekly topics and/or about your submissions. You are required to address your OnTrack reviewer's questions as a form of task discussion. Please frequently login to OnTrack for the task ***Discuss/Demonstrate*** or ***Resubmit*** equivalent to fix your work (if needed) based on the feedback to get your task signed as ***Complete***.